

DOSSIER DE PROJET

Entre 2 rives

Site de réservation de bateaux électriques sans permis

Jessie Gautherot

1. Introduction	4
2. Contexte et présentation de l'entreprise	4
2.1 Présentation de l'entreprise	4
2.2 Besoin et problématique	4
3. Analyse du besoin	5
3.1 Objectifs du projet	5
3.2 Publics concernés	5
3.3 Besoins fonctionnels	6
3.4 Règles métier	6
3.5 Éléments hors périmètre	7
4. Conception graphique et interfaces	8
4.1 Identité visuelle	8
4.2 Wireframes et conception responsive	9
5. Environnement et choix techniques	11
5.1 Environnement de développement et technologies utilisées	11
5.2 Choix techniques	12
5.3 Architecture générale	14
6. Conception de la base de données	14
6.1 Identification et organisation des données	14
6.2 Modélisation des données	15
6.3 Gestion de la disponibilité	16
7. Réalisation du front-end	16
7.1 Structure et organisation des interfaces	17
7.2 Mise en page et responsive	17
7.3 Réalisation d'une interface utilisateur statique avec Twig	19
7.4 Réalisation d'une interface utilisateur dynamique avec JavaScript	21
7.5 Prise en compte de l'accessibilité	22
7.6 Parcours utilisateur final	23
8. Réalisation du back-end	23
8.1 Traduction du modèle en entités Doctrine	23
8.2 Accès aux données avec les repositories	25

8.3 Traitement d'une requête côté serveur	26
9. Flux de données et logique métier de la réservation	27
9.1 Contrôle et création de la réservation	28
9.2 Paiement et confirmation de la réservation	30
9.3 Annulation et remboursement	32
10. Administration et gestion des données	34
10.1 Tableau de bord et accès à l'administration	34
10.2 Gestion des données et opérations d'administration	35
11. Sécurité et protection des données	37
11.1 Authentification et contrôle des accès	37
11.2 Sécurisation des actions et des données	38
11.3 Protection des données sensibles et personnelles	40
12. Référencement	41
13. Tests et validation	41
13.1 Stratégie et environnement de test	41
13.2 Test de la disponibilité d'un bateau	42
13.3 Test fonctionnel du parcours de réservation	43
13.4 Exécution et résultats des tests	44
14. Déploiement et documentation	45
14.1 Documentation du projet	45
14.2 Préparation au déploiement	46
15. Difficultés rencontrées	46
15.1 Compréhension et mise en place des webhooks Stripe en environnement local	46
16. Bilan du projet et conclusion	48
Annexes	49
Wireframes de la page d'accueil en format desktop, tablette, puis mobile.....	50
Wireframe du catalogue en format desktop, tablette puis mobile	53

1. Introduction

Entre 2 Rives est un projet personnel réalisé à l'issue de mes travaux de formation, dans le cadre de la préparation du titre professionnel Développeur Web et Web Mobile.

L'idée m'est venue pendant un week-end près du lac du Bourget. En passant devant une petite base de location de bateaux sans permis, nous avons décidé, sur un coup de tête, d'en louer un. J'ai vraiment adoré l'expérience.

C'est directement sur le bateau que le projet a commencé à prendre forme. Pendant la balade, je me suis demandé à quoi pouvait ressembler la base de données du loueur, comment étaient gérés les bateaux et les réservations. En repartant, je me suis dit que cette activité ferait un projet très intéressant à développer pour la préparation de mon diplôme.

J'ai ainsi imaginé Entre 2 Rives, une petite base nautique fictive, venant d'ouvrir dans le même village et proposant la location de bateaux électriques sans permis. L'objectif était de concevoir une application web complète reproduisant les principaux besoins d'une véritable activité de location.

L'application permet aux clients de découvrir les différents modèles de bateaux, de rechercher une disponibilité, d'effectuer une réservation et de payer leur location en ligne. Elle propose également un espace d'administration permettant au gestionnaire de gérer l'activité de la base nautique.

Ce dossier présente les différentes étapes de conception et de développement de l'application, depuis l'analyse du besoin jusqu'à sa réalisation, sa sécurisation, ses tests et la préparation de son déploiement.

2. Contexte et présentation de l'entreprise

2.1 Présentation de l'entreprise

Entre 2 Rives est une jeune entreprise fictive spécialisée dans la location de bateaux électriques accessibles sans permis.

L'entreprise propose plusieurs modèles de bateaux adaptés à différents usages et capacités d'accueil, de la balade en couple ou en famille jusqu'aux sorties en groupe.

Sa flotte est composée de trois modèles de bateaux. Chaque modèle est décliné en trois bateaux physiques afin de disposer de plusieurs embarcations disponibles à la location.

Les clients choisissent ainsi un modèle correspondant à leurs besoins et une durée de location, allant de quelques heures à la journée.

2.2 Besoin et problématique

Entre 2 Rives souhaite disposer d'un site web permettant de présenter son activité et de faciliter la réservation de ses bateaux.

L'entreprise a besoin d'une solution permettant aux visiteurs de consulter les modèles proposés, de rechercher leurs disponibilités et d'effectuer une réservation en ligne, tout en simplifiant la gestion quotidienne pour l'administrateur.

La principale problématique concerne la gestion des disponibilités. Le client réserve un modèle de bateau et non un bateau physique précis. L'application doit donc être capable de déterminer automatiquement si un bateau correspondant au modèle choisi est disponible selon les critères de la réservation, puis de lui attribuer une embarcation sans créer de chevauchement avec une réservation existante.

Le projet doit ainsi répondre à un double besoin : proposer aux clients un parcours de réservation simple et sécurisé, tout en fournissant à l'entreprise un outil permettant de gérer efficacement son activité.

3. Analyse du besoin

3.1 Objectifs du projet

Afin de répondre aux besoins identifiés, l'application doit permettre de :

- présenter l'entreprise, son activité et ses prestations ;
- présenter les différents modèles de bateaux proposés à la location ;
- rechercher les disponibilités selon les critères renseignés par l'utilisateur ;
- permettre la création et l'activation d'un compte client ;
- permettre la réservation d'un bateau et son paiement sécurisé en ligne ;
- permettre aux clients de consulter leurs informations personnelles et leurs réservations ;
- permettre l'annulation d'une réservation lorsque les conditions prévues sont respectées ;
- fournir un espace d'administration permettant de gérer les modèles, la disponibilité des bateaux physiques, les tarifs et les réservations, ainsi que de consulter les informations des utilisateurs.

L'objectif est de proposer une application simple et intuitive pour les clients, tout en répondant aux besoins de gestion de la base nautique.

3.2 Publics concernés

Visiteur

Personne non authentifiée qui souhaite découvrir l'activité, consulter les modèles de bateaux et préparer une éventuelle location.

Client

Utilisateur disposant d'un compte actif qui souhaite réserver et payer une location, puis consulter ses informations personnelles et retrouver ses réservations, avec la possibilité de demander leur annulation lorsque les conditions prévues sont respectées.

Administrateur

Gestionnaire de la base nautique qui a besoin d'administrer l'activité depuis un espace dédié.

3.3 Besoins fonctionnels

Pour le visiteur

- consulter les modèles de bateaux ;
- consulter les tarifs ;
- rechercher une disponibilité ;
- consulter les informations pratiques ;
- créer et activer un compte ;
- utiliser le formulaire de contact.

Pour le client

- se connecter à son compte ;
- consulter ses informations personnelles ;
- rechercher et réserver une location ;
- payer sa réservation en ligne ;
- consulter ses réservations ;
- demander l'annulation d'une réservation lorsque les conditions le permettent.

Pour l'administrateur

- consulter les réservations et leur état de paiement ;
- gérer les modèles de bateaux ;
- gérer la disponibilité des bateaux physiques ;
- gérer les tarifs ;
- gérer les réservations ;
- consulter les informations des utilisateurs.

3.4 Règles métier

Les principales règles métier ont été définies afin de garantir la cohérence des réservations, la disponibilité des bateaux et le respect des conditions de location.

Comptes utilisateurs

- Un compte nouvellement créé doit être activé par e-mail avant de pouvoir effectuer une réservation.
- Seul un utilisateur connecté disposant d'un compte actif peut réserver.

Modèles, bateaux et tarifs

- Le client réserve un modèle de bateau et non un bateau physique précis.
- Un bateau physique actif correspondant au modèle choisi est attribué automatiquement lors de la réservation.
- Le nombre de passagers ne peut pas dépasser la capacité maximale du modèle choisi.
- Le tarif sélectionné doit correspondre au modèle de bateau réservé.

Disponibilités et réservations

- Un même bateau physique ne peut pas être attribué à deux réservations dont les créneaux se chevauchent.
- Les locations sont possibles du mercredi au dimanche.
- Des créneaux horaires de location sont définis à l'avance et ne sont pas saisis librement par le client.
- Une réservation doit être effectuée au plus tard la veille de la location.
- Les conditions générales de location doivent être acceptées avant la validation de la réservation.

Paielement et annulation

- Une réservation n'est confirmée qu'après validation du paiement.
- Le client peut demander l'annulation d'une réservation confirmée jusqu'à 48 heures avant le début de la location.

3.5 Éléments hors périmètre

Le projet se concentre sur la présentation de l'activité ainsi que sur la réservation et la gestion des locations.

Certains besoins liés à l'exploitation réelle d'une base nautique ont volontairement été exclus du périmètre du projet, notamment :

- la gestion des états des lieux et des cautions ;
- la gestion des réparations et de l'entretien des bateaux ;
- la gestion de la comptabilité ;
- la gestion du niveau de charge des batteries et du temps de recharge nécessaire entre deux locations.

Dans le cadre du projet, ces éléments sont considérés comme étant gérés directement par la base nautique, indépendamment de l'application.

Ils pourraient toutefois faire l'objet d'évolutions ultérieures si l'application devait être adaptée aux besoins complets d'une entreprise réelle.

4. Conception graphique et interfaces

4.1 Identité visuelle

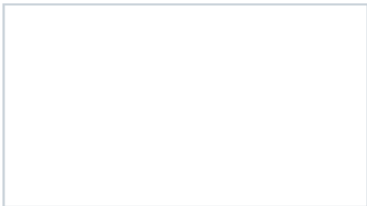
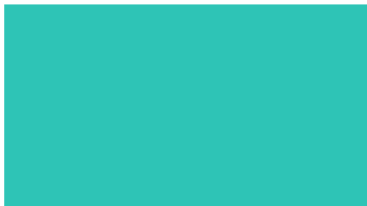
Logo



Logo d'Entre 2 Rives - Identité visuelle de l'entreprise

Palette de couleurs

L'identité visuelle d'Entre 2 Rives repose sur une palette volontairement limitée à trois couleurs principales afin de conserver une interface sobre et cohérente.

		
Bleu ardoise #334B5E	Blanc #FFFFFF	Turquoise #2EC4B6
Couleur principale de l'identité visuelle. Elle est utilisée pour structurer l'interface, notamment dans l'en-tête, le pied de page, les titres et certains éléments de navigation. Sa teinte sombre assure un bon contraste avec les fonds clairs tout en évoquant l'univers nautique.	Couleur de fond principale de l'interface. Elle apporte de la luminosité, facilite la lecture des contenus et laisse une place importante aux photographies des bateaux et des paysages.	Couleur d'accent destinée à attirer l'attention sur les éléments interactifs et les actions principales, notamment les boutons. Elle permet de les distinguer visuellement du reste de l'interface tout en rappelant l'univers de l'eau.

Palette de couleurs utilisée pour l'identité visuelle d'Entre 2 Rives

Typographies utilisées dans l'interface

Playfair Display pour l'identité de l'entreprise ;

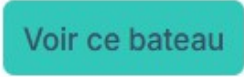


Entre 2 Rives

Inter pour les contenus.

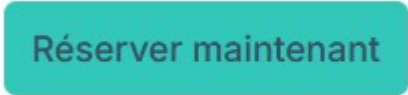
Venez découvrir les sublimes paysages du canal de Savières et du lac du Bourget. Prenez la barre d'un bateau électrique, sans permis et silencieux !

Boutons



Voir ce bateau

Bouton d'action standard



Réserver maintenant

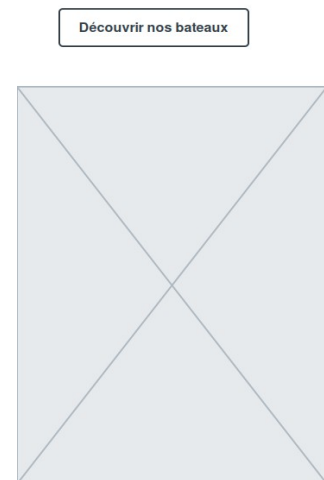
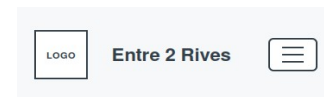
Bouton d'appel à l'action (CTA)

Principes graphiques

- Identité visuelle cohérente
- Interface sobre et lumineuse
- Navigation claire
- Mise en valeur des photographies
- Actions principales facilement identifiables

4.2 Wireframes et conception responsive

Les wireframes permettent de représenter la structure des interfaces ainsi que leur adaptation aux différentes tailles d'écran. Des versions desktop, tablette et mobile illustrent la manière dont l'organisation de la page évolue selon l'espace disponible. La disposition est progressivement réorganisée afin de conserver une interface lisible et fonctionnelle sur chaque type d'écran. Sur les écrans larges, l'espace disponible permet notamment de présenter certains contenus côte à côte et d'afficher directement les différents liens de navigation.



Wireframes de la page d'accueil – Versions desktop tablette et mobile

Lorsque la largeur diminue, les éléments sont redistribués : les contenus initialement disposés sur plusieurs colonnes passent progressivement sur une seule colonne, tandis que la navigation principale est regroupée dans un menu burger. Les différentes sections conservent cependant leur ordre et les actions principales restent facilement accessibles.

Ces adaptations permettent de préserver la hiérarchie de l'information et la cohérence visuelle de l'interface tout en tenant compte des contraintes propres aux formats desktop, tablette et mobile.

Les wireframes des pages d'accueil (page 50) et du catalogue (page 53) sont présentes en annexe dans leurs versions desktop, tablette et mobile,

5. Environnement et choix techniques

5.1 Environnement de développement et technologies utilisées

Le projet Entre 2 Rives a été développé en local sous Windows, avec Visual Studio Code. L'application repose principalement sur PHP 8.2 et Symfony 7, utilisé pour structurer la partie serveur selon une architecture MVC (Modèle-Vue-Contrôleur).

Environnement de développement

Visual Studio Code a été utilisé comme éditeur principal pour travailler sur les différents fichiers du projet : PHP, Twig, CSS, JavaScript et fichiers de configuration.

L'application est exécutée localement avec le serveur de développement Symfony CLI. Composer assure la gestion des dépendances PHP nécessaires au fonctionnement de l'application.

Git permet de suivre les modifications du code source. Un dépôt distant GitHub est également utilisé pour sauvegarder le projet et conserver l'historique des versions.

Technologies back-end

Le back-end est développé en PHP 8.2 avec Symfony 7. Le framework fournit la structure générale de l'application et prend notamment en charge les routes, les contrôleurs, les formulaires, l'authentification et différents mécanismes de sécurité.

Doctrine ORM assure les échanges avec la base de données MySQL 8. Celle-ci contient notamment les utilisateurs, les modèles de bateaux, les bateaux physiques, les tarifs de location, les réservations et les paiements.

Les migrations Doctrine permettent de faire évoluer la structure de la base de données lorsque les entités sont modifiées.

Twig est utilisé pour générer les vues HTML à partir des données transmises par les contrôleurs.

Technologies front-end

Les interfaces sont réalisées en HTML5, CSS3 et JavaScript.

Bootstrap est principalement utilisé pour la structure responsive des pages, notamment son système de grille et certains composants. Un CSS personnalisé complète cette structure pour appliquer l'identité visuelle d'Entre 2 Rives et définir les couleurs, les typographies, les espacements et l'apparence des composants.

JavaScript apporte des comportements dynamiques sans rechargement complet de la page. Il intervient notamment dans la recherche de bateaux disponibles et dans la récupération des tarifs proposés lors d'une réservation. Les données sont échangées avec le serveur au format JSON, puis JavaScript met à jour les éléments concernés dans le DOM.

Services et outils complémentaires

Stripe est intégré à l'application pour gérer le paiement en ligne des réservations ainsi que les remboursements. En environnement local, Stripe CLI transmet les événements Stripe au webhook de l'application pour tester les échanges avec le service de paiement.

L'application utilise également le composant d'envoi d'e-mails de Symfony, notamment pour le formulaire de contact.

Les principales fonctionnalités sont couvertes par des tests automatisés avec PHPUnit. Ces tests vérifient notamment le fonctionnement des repositories, des services de disponibilité et de réservation, du processus d'annulation et de l'intégration du paiement.

La configuration propre à chaque environnement est gérée avec des variables d'environnement, notamment pour la connexion à la base de données et les clés nécessaires aux services externes.

5.2 Choix techniques

Les technologies utilisées pour le développement d'Entre 2 Rives ont été choisies en fonction des besoins du projet, mais également des compétences déjà acquises.

L'objectif était de construire une application structurée et maintenable, en utilisant des outils adaptés à ses principales fonctionnalités : gestion des utilisateurs, catalogue de bateaux, recherche de disponibilités, réservation, paiement en ligne et administration.

Symfony

Symfony a été choisi pour structurer la partie back-end de l'application. Son architecture permet de séparer les différentes responsabilités : les contrôleurs gèrent principalement les requêtes et les réponses, les services prennent en charge la logique métier et les repositories assurent l'accès aux données.

Le framework fournit également plusieurs composants nécessaires au projet, notamment pour la gestion des routes, des formulaires, de l'authentification, des autorisations et de la protection CSRF.

Ce choix repose aussi sur les compétences acquises pendant la formation. Ma maîtrise de Symfony et de PHP m'a permis de me concentrer davantage sur la conception et les fonctionnalités propres à Entre 2 Rives, plutôt que sur l'apprentissage d'un nouveau framework.

Doctrine et MySQL

Doctrine ORM est utilisé pour assurer les échanges entre l'application Symfony et la base de données MySQL. Il permet de représenter les données sous forme d'entités PHP et de définir leurs relations.

Cette approche correspond à la structure du projet, dont les principales entités sont liées entre elles : utilisateurs, modèles de bateaux, bateaux physiques, tarifs de location, réservations et paiements.

Les repositories regroupent les requêtes spécifiques nécessaires à l'application, notamment celles liées à la recherche des disponibilités. Les migrations Doctrine permettent de faire évoluer la structure de la base de données lorsque les entités sont modifiées.

MySQL a été choisi comme système de gestion de base de données relationnelle. Il est adapté aux données structurées du projet et permet de définir les relations et les contraintes nécessaires à leur cohérence.

Twig

Twig est utilisé comme moteur de templates pour générer les pages HTML à partir des données transmises par les contrôleurs Symfony.

Bootstrap et CSS personnalisé

Bootstrap a été retenu principalement pour son système de grille et ses outils de mise en page responsive qui permettent de réorganiser les contenus en fonction de la largeur de l'écran.

Son utilisation reste volontairement limitée aux besoins structurels des interfaces. Le CSS personnalisé prend en charge l'identité visuelle d'Entre 2 Rives, notamment les couleurs, les typographies, les espacements et l'apparence des composants.

Cette séparation permet de profiter des outils responsives de Bootstrap tout en conservant la maîtrise de l'apparence du site, sans dépendre de ses styles par défaut.

JavaScript et Fetch

JavaScript est utilisé lorsque certaines parties de l'interface nécessitent une mise à jour dynamique. Le reste de l'application repose principalement sur le rendu des pages côté serveur avec Symfony et Twig.

L'API Fetch permet d'envoyer des requêtes asynchrones au serveur, notamment pour rechercher les bateaux ou les tarifs disponibles. Le serveur retourne les résultats au format JSON, puis JavaScript met à jour les éléments concernés dans le DOM, sans recharger toute la page.

Stripe

Stripe a été choisi pour gérer les paiements en ligne.

Lors d'une réservation, l'application crée une session de paiement et redirige le client vers l'interface sécurisée de Stripe. Les webhooks permettent ensuite de recevoir les événements liés au paiement et de mettre à jour les états correspondants dans Entre 2 Rives.

Stripe intervient également dans le processus de remboursement lorsqu'une réservation est annulée.

Ce choix permet de confier le traitement des données bancaires à un service spécialisé, tout en conservant dans l'application la gestion des réservations, des paiements et de leurs différents états.

5.3 Architecture générale

Entre 2 Rives repose principalement sur une architecture MVC (Modèle-Vue-Contrôleur). Cette architecture sépare les responsabilités des différents composants de l'application.

- Le modèle représente les données manipulées par l'application. Il est notamment constitué des entités Doctrine User, BoatModel, Boat, RentalRate, Booking et Payment.
- Les vues, réalisées avec Twig, génèrent les interfaces HTML à partir des données transmises par l'application.
- Les contrôleurs reçoivent les requêtes HTTP et coordonnent leur traitement. Selon l'action demandée, ils peuvent retourner une vue, effectuer une redirection ou renvoyer des données au format JSON.

Cette architecture est complétée par des repositories et des services, qui assurent une séparation plus précise des responsabilités.

Les repositories regroupent les opérations d'accès aux données qui nécessitent des requêtes spécifiques. Ils sont notamment utilisés pour rechercher les informations enregistrées en base de données et vérifier l'existence de réservations bloquantes.

Les services contiennent la logique métier de l'application. Ils prennent notamment en charge la recherche des disponibilités, la création des réservations et les traitements liés aux paiements.

Cette organisation permet de conserver des contrôleurs centrés sur la gestion des requêtes et des réponses, sans y placer l'ensemble des traitements métier. Elle améliore la lisibilité et la maintenabilité du code, tout en facilitant les tests des différents composants.

6. Conception de la base de données

6.1 Identification et organisation des données

L'analyse des besoins et des règles métier m'a permis d'identifier les principales données nécessaires au fonctionnement de l'application.

Le modèle de données repose principalement sur :

- **User** : représente les utilisateurs de l'application, clients et administrateurs ;
- **BoatModel** : représente les modèles de bateaux proposés à la location ;
- **Boat** : représente les bateaux physiques appartenant à la flotte ;
- **RentalRate** : représente les créneaux et les tarifs proposés pour les différents modèles ;
- **Booking** : représente les réservations effectuées par les clients ;
- **Payment** : représente les paiements associés aux réservations.

Distinction entre modèle et bateau physique

Une distinction a été faite entre un modèle de bateau et un bateau physique.

Par exemple, Escapade représente un modèle proposé au client, tandis que Escapade 01, Escapade 02 et Escapade 03 représentent les différentes embarcations physiques correspondant à ce modèle.

Cette séparation permet au client de choisir le modèle correspondant à ses besoins et à l'application de lui attribuer un bateau physique appartenant au modèle choisi en fonction des disponibilités.

Relations entre les données

L'organisation des données repose sur les liens nécessaires au fonctionnement de la réservation.

Un modèle de bateau peut regrouper plusieurs bateaux physiques et proposer plusieurs tarifs de location. Chaque bateau et chaque tarif sont rattachés à un seul modèle de bateau.

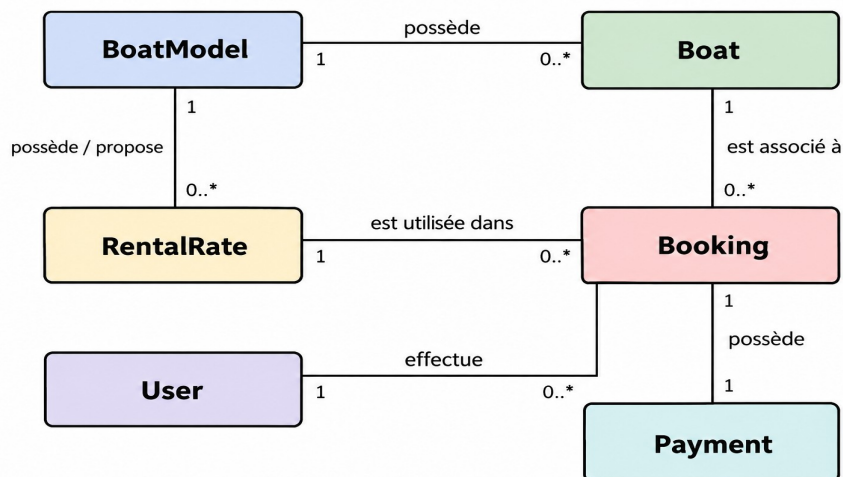
Un utilisateur peut effectuer plusieurs réservations, tandis qu'une réservation appartient à un seul utilisateur. Chaque réservation est également associée à un modèle de bateau, un bateau physique et un tarif de location. Un même bateau et un même tarif peuvent être utilisés dans plusieurs réservations au cours du temps.

Enfin, chaque réservation est associée à un paiement, et chaque paiement correspond à une seule réservation.

Ces relations permettent de représenter les liens entre les principales données nécessaires au fonctionnement de l'application et d'en définir les cardinalités.

6.2 Modélisation des données

Afin de représenter l'organisation des données définie précédemment, un schéma a été réalisé pour visualiser les principales entités de l'application ainsi que les relations et les cardinalités qui les relient.



Modélisation des principales données de l'application et de leurs relations

Cette modélisation met notamment en évidence la distinction entre les modèles de bateaux et les embarcations physiques, ainsi que la place centrale de la réservation, qui relie l'utilisateur, le modèle choisi, le bateau attribué et le tarif sélectionné.

6.3 Gestion de la disponibilité

La disponibilité d'un bateau ne peut pas être stockée comme une information fixe, car elle évolue en fonction des réservations. Un bateau actuellement en location peut par exemple être disponible pour une réservation prévue plus tard dans la journée ou à une date future.

Il est donc nécessaire de déterminer sa disponibilité pour une période donnée. Pour cela, plusieurs informations doivent être prises en compte : le bateau doit être actif et ne pas être déjà réservé pendant cette période.

Il faut donc comparer la période de location aux réservations déjà enregistrées pour ce bateau, afin de vérifier s'il est disponible.

Les besoins métier prévoient des créneaux de location définis à l'avance. Ce choix permet au loueur de maîtriser les horaires de départ et de retour des bateaux, tout en limitant le nombre de possibilités à gérer dans l'application. J'ai choisi de traduire ce fonctionnement dans les données en associant les créneaux aux tarifs de location dans l'entité `RentalRate`.

La logique de disponibilité pourra alors s'appuyer sur ces créneaux prédéfinis pour définir les périodes de location à comparer avec les réservations existantes.

7. Réalisation du front-end

Les interfaces de l'application Entre 2 Rives ont été réalisées avec Twig, Bootstrap, CSS et JavaScript. Elles respectent l'identité graphique définie précédemment et s'adaptent aux différentes tailles d'écran.

Le développement front-end repose également sur la réutilisation des éléments communs, la prise en compte de l'accessibilité et l'ajout d'interactions dynamiques lorsque certaines parties de

l'interface doivent être mises à jour sans recharger entièrement la page.

7.1 Structure et organisation des interfaces

Les interfaces de l'application sont générées avec Twig. Elles utilisent un système d'héritage de templates pour éviter de répéter le code commun aux différentes pages.

Le fichier `base.html.twig` définit la structure principale de l'interface. Il contient les éléments communs ainsi que les blocs que les templates enfants peuvent compléter ou redéfinir. Chaque page peut ainsi définir son titre, ses feuilles de style et son contenu tout en conservant la même structure générale.

L'en-tête et le pied de page sont placés dans des templates dédiés puis intégrés à cette structure. Ils ne sont donc pas dupliqués dans chaque page et une modification est directement appliquée à l'ensemble de l'application.

Une feuille de style principale regroupe les règles communes à l'application, comme les couleurs, les typographies, les boutons et les éléments partagés. Des feuilles de style spécifiques complètent cette base pour répondre aux besoins propres à chaque interface.

Les templates utilisent également des balises HTML sémantiques pour structurer les différents contenus, notamment la navigation, les sections principales et le pied de page. Ces balises rendent la structure des pages plus claire et participent également à leur accessibilité.

7.2 Mise en page et responsive

La mise en page des interfaces repose sur une combinaison de Bootstrap et de CSS personnalisé. Bootstrap est principalement utilisé pour structurer les pages avec son système de grille, tandis que le CSS personnalisé permet d'appliquer l'identité visuelle d'Entre 2 Rives.

Le système de grille de Bootstrap adapte la disposition des contenus à la largeur de l'écran grâce à différents breakpoints. Les colonnes peuvent ainsi être réorganisées ou changer de largeur selon l'espace disponible. Flexbox complète cette structure pour gérer plus précisément l'alignement et la disposition des éléments au sein des composants.

Le CSS personnalisé prend en charge les éléments propres à l'identité du site, notamment les couleurs, les typographies, les espacements et l'apparence des différents composants. Cette séparation permet d'utiliser Bootstrap pour la structure des interfaces sans dépendre entièrement de ses styles visuels.

La section « Prêt à larguer les amarres ? » de la page d'accueil illustre concrètement l'adaptation de cette structure aux différentes tailles d'écran.

Sur les écrans larges, l'image et le contenu textuel sont disposés côte à côte. Les classes `col-xl-7` et `col-xl-5` répartissent alors l'espace entre les deux blocs. Sur les écrans plus étroits, la classe `col-12` leur fait occuper toute la largeur disponible : l'image et le contenu sont alors affichés l'un sous l'autre.

```

<div class="row align-items-center g-5">
  <div class="col-12 col-xl-7">
    [...]
  </div>

  <div class="col-12 col-xl-5">
    [...]
  </div>
</div>

```

home/index.html.twig – Adaptation de la disposition avec la grille Bootstrap



Prêt à larguer les amarres ?

Choisissez votre date, votre bateau et la formule qui vous convient pour profiter du lac et du canal.

Réserver maintenant



Prêt à larguer les amarres ?

Choisissez votre date, votre bateau et la formule qui vous convient pour profiter du lac et du canal.

Réserver maintenant



Prêt à larguer les amarres ?

Choisissez votre date, votre bateau et la formule qui vous convient pour profiter du lac et du canal.

Réserver maintenant

Adaptation responsive de la section « Prêt à larguer les amarres ? » - Versions desktop, tablette et mobile

Les classes responsives de Bootstrap permettent ainsi de gérer les principaux changements de disposition. Elles sont complétées par des media queries lorsque certains éléments nécessitent une adaptation plus précise.

C'est notamment le cas de l'image de cette section. Sa hauteur est adaptée à plusieurs largeurs d'écran afin de conserver un cadrage satisfaisant malgré les changements de proportions de son conteneur. Sur les formats intermédiaires, l'image occupe toute la largeur de la grille et nécessite une hauteur plus importante. À partir du breakpoint xl, elle est de nouveau placée dans une colonne de sept unités et sa hauteur est ajustée en conséquence.

```
.home-booking-picture img {  
  display: block;  
  width: 100%;  
  height: 250px;  
  object-fit: cover;  
}  
  
@media (min-width: 576px) {  
  .home-booking-picture img {  
    height: 350px;  
  }  
}  
  
@media (min-width: 992px) {  
  .home-booking-picture img {  
    height: 450px;  
  }  
}  
  
@media (min-width: 1200px) {  
  .home-booking-picture img {  
    height: 400px;  
  }  
}
```

home.css - Adaptation du cadrage de l'image selon la largeur de l'écran

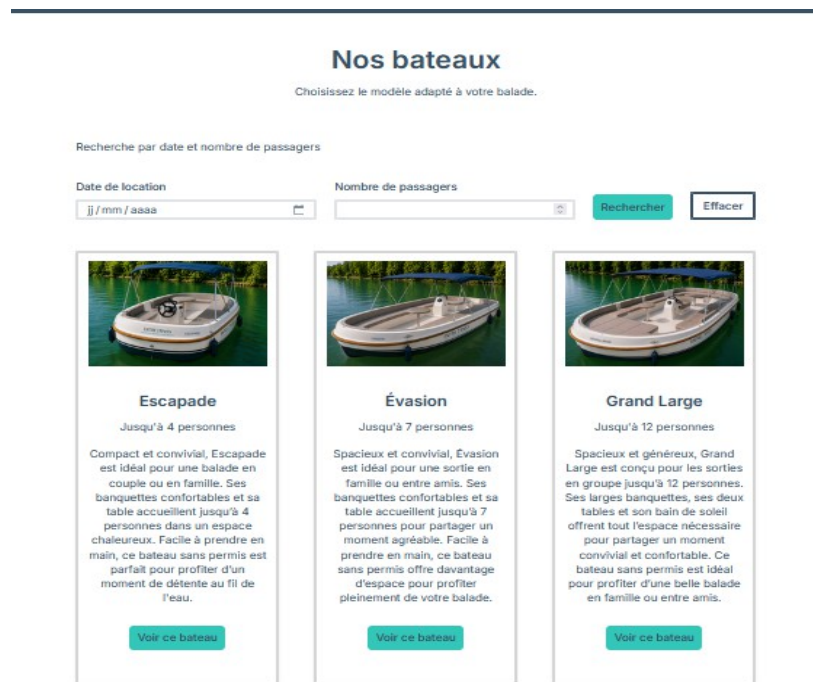
Ces différents mécanismes permettent de faire évoluer la disposition et la présentation des interfaces selon l'espace disponible, tout en conservant la lisibilité des contenus et l'accès aux principales actions.

7.3 Réalisation d'une interface utilisateur statique avec Twig

Les templates Twig permettent de construire le HTML des interfaces à partir des données transmises par l'application. Le catalogue des bateaux illustre ce fonctionnement : les modèles de bateaux sont récupérés par le back-end puis transmis au template afin de générer leur affichage.

La page associe un formulaire de recherche par date et nombre de passagers à la présentation des différents modèles de bateaux. Dans cette partie, l'exemple présenté se concentre sur la réalisation de l'interface initiale du catalogue en HTML, CSS et Twig, ainsi que sur son adaptation responsive. Les données des modèles sont intégrées au HTML lors du rendu côté serveur. Le développement d'une interface dynamique avec JavaScript est présenté séparément dans la section suivante.

Chaque modèle est affiché sous la forme d'une carte contenant son image principale, son nom, sa capacité, sa description et un lien permettant d'accéder à sa fiche détaillée.



Catalogue des modèles de bateaux - Version desktop

L'interface s'adapte également à la largeur de l'écran grâce à la grille Bootstrap. Les cartes occupent toute la largeur sur les petits écrans et sont réparties sur trois colonnes à partir du breakpoint lg.

```

>
{% for boatModel in boatModels %}
  <div class="col-12 col-lg-4">
    <article class="card h-100 shadow-sm">
      
      <div class="card-body text-center d-flex flex-column">
        <h2 class="card-title boat-catalogue-card-title">
          {{ boatModel.name }}
        </h2>
        <p class="boat-catalogue-capacity">
          Jusqu'à {{ boatModel.capacity }} personnes
        </p>
        <p class="card-text">
          {{ boatModel.description }}
        </p>
        <a
          class="btn btn-primary-custom boat-catalogue-card-button align-self-center"
          href="{{ path('app_boat_show', {slug: boatModel.slug}) }}"
          aria-label="Voir le bateau {{ boatModel.name }}"
        >
          Voir ce bateau
        </a>
      </div>
    </article>
  </div>
{% else %}
  <div class="text-center">

```

boat/index.html.twig - Génération des cartes du catalogue avec Twig

La collection `boatModels`, transmise au template par le back-end, est parcourue avec une boucle `{% for %}`. À chaque itération, Twig utilise les propriétés du modèle courant pour renseigner les différents éléments de la carte : `mainImage`, `name`, `capacity` et `description`.

Le lien vers la fiche détaillée est également généré avec la fonction `path()`. Le slug du modèle courant est transmis à la route `app_boat_show`, ce qui permet à chaque carte de diriger l'utilisateur vers la fiche correspondante.

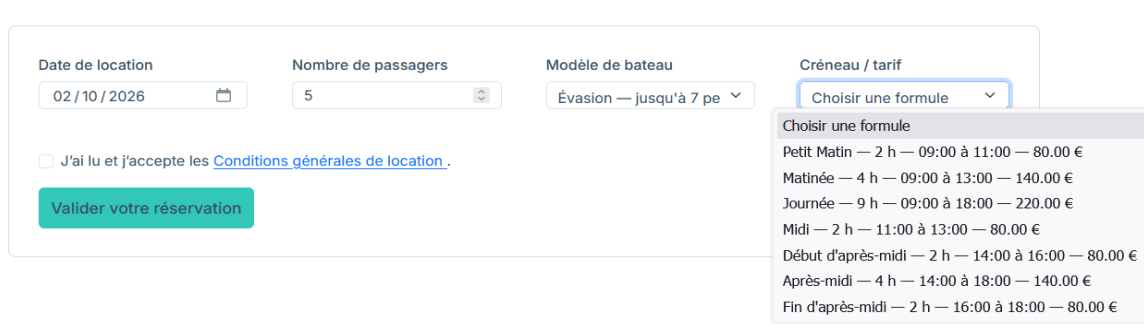
La structure `{% for %} ... {% else %}` permet enfin de prévoir le cas où aucun modèle n'est présent dans la collection : un message est alors affiché à la place des cartes.

Certains attributs d'accessibilité sont eux aussi construits à partir des données du modèle. Le texte alternatif de l'image et l'attribut `aria-label` du lien intègrent notamment le nom du bateau afin de donner davantage de contexte à l'utilisateur.

7.4 Réalisation d'une interface utilisateur dynamique avec JavaScript

JavaScript est utilisé pour apporter des interactions dynamiques à certaines interfaces sans nécessiter le rechargement complet de la page. Le formulaire de réservation en constitue un exemple avec la mise à jour des formules de location disponibles.

Lorsque l'utilisateur modifie la date ou le modèle de bateau, la fonction `loadAvailableRentalRates()` est déclenchée par des écouteurs d'événements `change`. La recherche n'est cependant exécutée que lorsque ces deux informations sont renseignées.



Mise à jour dynamique des tarifs de location disponibles dans le formulaire de réservation

La liste affichée dans le champ « Créneau/tarif » correspond aux données récupérées dynamiquement en fonction de la date et du modèle sélectionnés.

```
// Waits until a date and a boat model are selected.
if (!date || !modelId) {
  return;
}

try {
  // Gets the available rental rates from the server.
  const response = await fetch(
    `/reservation/formules-disponibles?date=${date}&model=${modelId}`
  );

  if (!response.ok) {
    availabilityStatus.textContent =
      'Impossible de charger les créneaux disponibles.';
    return;
  }

  const rentalRates = await response.json();

  if (rentalRates.length === 0) {
    availabilityStatus.textContent =
      'Aucun créneau disponible pour cette date.';
    return;
  }
}
```

booking.js - Récupération asynchrone des tarifs disponibles selon la date et le modèle sélectionnés

La requête est envoyée de manière asynchrone avec l'API Fetch vers l'endpoint /reservation/formules-disponibles. La date et l'identifiant du modèle sélectionné sont transmis comme paramètres de la requête.

Le serveur détermine les tarifs de location disponibles correspondant à ces critères et retourne les résultats au format JSON. JavaScript vérifie la réponse avec `response.ok` avant de récupérer les données avec `response.json()`. L'absence de résultat et les éventuelles erreurs sont également prises en compte afin d'informer l'utilisateur.

```
// Adds each available rental rate to the select.
rentalRates.forEach((rentalRate) => {
  const option = document.createElement('option');

  option.value = rentalRate.id;

  option.textContent =
    `${rentalRate.label} - ` +
    `${rentalRate.durationHours} h - ` +
    `${rentalRate.startTime} à ${rentalRate.endTime} - ` +
    `${(rentalRate.price / 100).toFixed(2)} €`;

  rentalRateSelect.appendChild(option);
});

availabilityStatus.textContent =
  `${rentalRates.length} créneau(x) disponible(s).`;
```

booking.js - Création et ajout des tarifs disponibles dans le champ de sélection

Avant l'affichage des nouveaux résultats, les tarifs précédemment proposés sont retirés du champ de sélection avec `replaceChildren()`. Cela permet notamment d'éviter de conserver un tarif correspondant à une date ou à un modèle qui vient d'être modifié.

Les tarifs retournés sont parcourus avec `forEach()`. Pour chacun d'eux, JavaScript crée un élément `<option>` avec `createElement()`, lui associe l'identifiant du tarif et renseigne son contenu avec `textContent` à partir des données reçues : libellé, durée, horaires et prix. L'élément est ensuite ajouté au champ de sélection avec `appendChild()`, ce qui permet de mettre à jour dynamiquement le DOM.

Seule la liste des tarifs est ainsi actualisée : la structure générale du formulaire reste générée par Twig. Cette approche permet d'adapter les choix proposés à l'utilisateur en fonction de la date et du modèle sélectionnés, sans recharger l'ensemble de la page.

Un message indiquant l'état de la disponibilité est également mis à jour dynamiquement. Dans le template Twig, cette zone utilise l'attribut `aria-live="polite"` afin que son changement puisse être annoncé par les technologies d'assistance.

7.5 Prise en compte de l'accessibilité

L'accessibilité a été prise en compte dans la réalisation des différentes interfaces. La structure des pages utilise des éléments HTML sémantiques et les images disposent de textes alternatifs adaptés à leur contenu. Les champs de formulaire sont associés à des `label` et les groupes de champs sont structurés lorsque nécessaire avec `fieldset` et `legend`.

Des attributs ARIA complètent ces éléments lorsque le HTML seul ne suffit pas à transmettre l'information. `aria-label` apporte notamment du contexte à certains liens ou éléments de navigation, `aria-current="page"` permet d'identifier la page active et `aria-live="polite"` est utilisé pour annoncer les changements de contenu produits dynamiquement.

Les éléments interactifs disposent également d'états de focus visibles. Enfin, les erreurs de validation et les messages affichés après certaines actions permettent de fournir à l'utilisateur un retour sur le résultat de ses interactions.

7.6 Parcours utilisateur final

Le parcours principal de l'utilisateur permet de relier les différentes interfaces réalisées au sein d'un même processus de réservation. Depuis la page d'accueil, l'utilisateur peut consulter le catalogue, accéder à la fiche d'un modèle, renseigner les informations nécessaires à sa réservation puis procéder au paiement avant d'obtenir la confirmation de celle-ci.



Parcours principal de l'utilisateur lors d'une réservation

8. Réalisation du back-end

8.1 Traduction du modèle en entités Doctrine

Le modèle de données défini lors de la phase de conception a été traduit dans l'application à l'aide de Doctrine ORM. Les données sont représentées par des entités PHP dont les propriétés et les associations permettent de définir leur persistance dans la base de données MySQL.

L'entité Booking constitue un exemple représentatif de cette mise en œuvre. Elle possède son propre identifiant et est associée aux différentes données nécessaires à une réservation : l'utilisateur, le modèle de bateau choisi, le bateau physique attribué, le tarif de location et le paiement.

```

#[ORM\Id]
#[ORM\GeneratedValue]
#[ORM\Column]
private ?int $id = null;

#[ORM\ManyToOne(inversedBy: 'bookings')]
#[ORM\JoinColumn(nullable: false)]
private ?User $user = null;

#[ORM\ManyToOne]
#[ORM\JoinColumn(nullable: false)]
private ?BoatModel $boatModel = null;

#[ORM\ManyToOne(inversedBy: 'bookings')]
#[ORM\JoinColumn(nullable: false)]
private ?Boat $boat = null;

#[ORM\ManyToOne]
#[ORM\JoinColumn(nullable: false)]
private ?RentalRate $rentalRate = null;

#[ORM\OneToOne(mappedBy: 'booking')]
private ?Payment $payment = null;

#[ORM\Column]
#[Assert\Positive(
    message: 'Le nombre de passagers doit être supérieur à zéro.'
)]
private ?int $passengerCount = null;

```

Booking.php - Définition des propriétés et des relations de l'entité avec Doctrine

L'identifiant de la réservation est défini avec les attributs `ORM\Id`, `ORM\GeneratedValue` et `ORM\Column`. Sa valeur est ainsi générée automatiquement lors de l'enregistrement d'une nouvelle réservation.

Les associations avec `User`, `BoatModel`, `Boat` et `RentalRate` sont définies avec `ORM\ManyToOne`. L'attribut `ORM\JoinColumn(nullable: false)` indique que ces relations sont obligatoires : une réservation enregistrée doit être associée à chacun de ces éléments. Ces associations se traduisent dans la base de données par des clés étrangères, qui permettent de relier la table `booking` aux tables correspondantes.

La relation avec `Payment` utilise quant à elle `ORM\OneToOne`, puisqu'une réservation est associée à un seul paiement et qu'un paiement correspond à une seule réservation.

Doctrine permet également de définir les propriétés directement enregistrées dans la table. Dans cet extrait, `passengerCount` est ainsi déclaré avec `ORM\Column`. Les contraintes de validation Symfony peuvent compléter les contraintes liées à la persistance : `Assert\Positive` vérifie ici, au niveau applicatif, que le nombre de passagers est supérieur à zéro.

Migrations

La structure définie par les entités Doctrine est répercutée dans la base de données à l'aide des migrations. Lorsqu'une modification du modèle nécessite une évolution du schéma, une migration est générée puis exécutée afin d'appliquer les modifications à la base MySQL.

```
php bin/console make:migration
```

```
php bin/console doctrine:migrations:migrate
```

La migration générée contient les instructions SQL nécessaires à la modification du schéma. Ce mécanisme permet de conserver une trace des évolutions de la structure de la base de données et de les appliquer de manière reproductible.

8.2 Accès aux données avec les repositories

Les repositories regroupent les opérations permettant d'interroger les données enregistrées dans la base. Doctrine fournit des méthodes de recherche courantes, comme `find()`, `findOneBy()` ou `findBy()`, mais permet également de créer des requêtes personnalisées lorsque les besoins de l'application sont plus spécifiques.

Cette organisation permet de séparer l'accès aux données des traitements métier réalisés dans les services. Le repository se charge d'interroger la base et de retourner le résultat nécessaire, tandis que les services utilisent ensuite ces informations pour appliquer les règles de l'application.

Dans Entre 2 Rives, cette séparation est notamment utilisée pour la gestion des disponibilités. Avant d'attribuer un bateau à une réservation, l'application doit déterminer si celui-ci possède déjà une réservation bloquante sur le créneau demandé. Cette recherche est réalisée par la méthode `hasBlockingBooking()` du `BookingRepository`.

```
*/
public function hasBlockingBooking(
    Boat $boat,
    \DateTimeImmutable $startAt,
    \DateTimeImmutable $endAt,
): bool {
    $count = $this->createQueryBuilder('booking')
        ->select('COUNT(booking.id)')
        ->andWhere('booking.boat = :boat')
        ->andWhere('booking.status IN (:blockingStatuses)')
        ->andWhere('booking.startAt < :endAt')
        ->andWhere('booking.endAt > :startAt')
        ->setParameter('boat', $boat)
        ->setParameter('blockingStatuses', [
            Booking::STATUS_PENDING,
            Booking::STATUS_CONFIRMED,
            Booking::STATUS_CANCELLATION_PENDING,
        ])
        ->setParameter('startAt', $startAt)
        ->setParameter('endAt', $endAt)
        ->getQuery()
        ->getSingleScalarResult();

    return $count > 0;
}
```

BookingRepository - hasBlockingBooking() - Recherche d'une réservation bloquante

La méthode reçoit le bateau physique concerné ainsi que les dates et heures de début et de fin du créneau à vérifier. La requête est construite avec le `QueryBuilder` de Doctrine et compte les réservations correspondant aux différents critères.

La recherche porte uniquement sur le bateau concerné et sur les statuts considérés comme bloquants : `PENDING`, `CONFIRMED` et `CANCELLATION_PENDING`. Une réservation annulée n'empêche donc pas le bateau d'être proposé sur un nouveau créneau.

La détection d'un chevauchement repose sur deux conditions :

```
booking.startAt < :endAt
```

```
booking.endAt > :startAt
```

Elles permettent de rechercher une réservation existante dont la période recoupe le créneau demandé. Si le nombre de réservations correspondant à ces critères est supérieur à zéro, la méthode retourne true ; dans le cas contraire, elle retourne false.

Le repository fournit ainsi une réponse sur les données présentes en base, sans prendre en charge l'ensemble de la logique de disponibilité. Cette responsabilité reste séparée des traitements métier qui utilisent ensuite ce résultat.

8.3 Traitement d'une requête côté serveur

Dans Entre 2 Rives, la soumission du formulaire de réservation permet d'illustrer le traitement d'une requête côté serveur. Lorsque l'utilisateur valide le formulaire, les données sont envoyées à l'application par une requête HTTP POST.

La route Symfony associée à cette requête est définie directement dans le BookingController :

```
/**
 * Creates a booking from the submitted booking form.
 */
#[Route('/reservation', name: 'app_booking_create', methods: ['POST'])]
public function create(
    Request $request,
    BoatModelRepository $boatModelRepository,
    RentalRateRepository $rentalRateRepository,
    BookingService $bookingService,
    StripeService $stripeService,
): Response {
```

BookingController - create() - Définition de la route de création d'une réservation

L'attribut Route associe l'URL /reservation à l'action create() du contrôleur. La propriété methods: ['POST'] précise que cette action est destinée au traitement des données envoyées par le formulaire.

Le contrôleur constitue ainsi le point d'entrée du traitement côté serveur : il reçoit la requête et coordonne les différentes étapes nécessaires avant de déléguer la logique métier aux services concernés.

Après avoir reçu la requête, le contrôleur vérifie d'abord le jeton CSRF afin de protéger la soumission du formulaire. Il récupère ensuite les différentes valeurs envoyées par l'utilisateur à partir de l'objet Request.

```
// Checks the CSRF token before processing the submitted form.
if (!$this->isCsrfTokenValid(
    'create_booking',
    $request->request->get('_token'),
)) {
    throw $this->createAccessDeniedException(
        'Jeton CSRF invalide.',
    );
}

$dateValue = $request->request->get('date');
$passengerCount = $request->request->getInt('passengerCount');
$boatModelId = $request->request->getInt('boatModel');
$rentalRateId = $request->request->getInt('rentalRate');
$acceptTerms = $request->request->getBoolean('acceptTerms');
```

BookingController - create() - Vérification du jeton CSRF et récupération des données de la requête

Les données nécessaires à la réservation sont ainsi récupérées côté serveur. Le contrôleur effectue ensuite de premiers contrôles, notamment sur la présence des informations obligatoires, l'acceptation des conditions générales et la validité des valeurs reçues. En cas d'erreur, un message est enregistré puis l'utilisateur est redirigé vers le formulaire.

Une fois les données récupérées et les premiers contrôles effectués, le contrôleur récupère l'utilisateur connecté puis transmet les informations nécessaires au BookingService.

```
$user = $this->getUser();

// Delegates all booking business rules and creation to BookingService.
try {
    $booking = $bookingService->makeBooking(
        $user,
        $boatModel,
        $rentalRate,
        $date,
        $passengerCount,
    );
} catch (\LogicException $exception) {
    $this->addFlash(
        'danger',
        $exception->getMessage(),
    );

    return $this->redirectToRoute('app_booking_new');
}

// Creates the Stripe Checkout Session for the new booking.
$checkoutUrl = $stripeService->createCheckoutSession($booking);

// Redirects the customer to the Stripe Checkout page.
return $this->redirect($checkoutUrl);
```

BookingController – create()- Délégation de la création de la réservation à BookingService

L'appel à makeBooking() permet de déléguer au service la logique métier liée à la réservation, plutôt que de la réaliser directement dans le contrôleur. Si une règle métier empêche la création de la réservation, l'exception est interceptée par le contrôleur, qui affiche un message à l'utilisateur et le redirige vers le formulaire.

Les règles appliquées par BookingService et la création de la réservation seront détaillées dans la partie 9.1.

Lorsque la réservation a été créée, le contrôleur prépare la suite du processus de paiement à l'aide de StripeService, puis retourne une redirection vers la page de paiement Stripe.

9. Flux de données et logique métier de la réservation

Cette partie présente les principales étapes du traitement d'une réservation, depuis la vérification des règles métier et sa création jusqu'au paiement, puis à son éventuelle annulation et à son remboursement.

9.1 Contrôle et création de la réservation

Une fois les données du formulaire récupérées et les premiers contrôles effectués par BookingController, la demande est transmise à la méthode BookingService::makeBooking(). BookingService prend alors en charge la logique métier nécessaire à la création de la réservation.

Avant de créer celle-ci, BookingService::makeBooking() appelle la méthode privée BookingService::checkBooking(), chargée de vérifier que la demande respecte les principales règles définies par l'application.

```
// Check the booking request.
$this->checkBooking([
    $user,
    $boatModel,
    $rentalRate,
    $date,
    $passengerCount,
]);
```

BookingService::checkBooking() - Vérification des règles métier de la réservation

Ces contrôles portent notamment sur l'activation du compte utilisateur et de l'offre choisie, la correspondance entre l'offre et le modèle de bateau, le respect de la capacité du bateau, ainsi que la date et le jour de réservation.

Si l'une de ces règles n'est pas respectée, une exception est déclenchée. Comme présenté précédemment, celle-ci est récupérée par BookingController afin d'informer l'utilisateur et de le rediriger vers le formulaire.

Est-ce qu'un bateau est réellement disponible ?

Une fois ces règles vérifiées, la réservation n'est pas encore créée. BookingService doit d'abord trouver un bateau physique disponible correspondant au modèle et au créneau demandés :

```
// Find an available physical boat.
$boat = $this->availabilityService->findAvailableBoat(
    $boatModel,
    $rentalRate,
    $date,
);

if ($boat === null) {
    throw new \LogicException(
        'Aucun bateau n'est disponible pour ce créneau.'
    );
}
```

BookingService::makeBooking() - Recherche d'un bateau physique disponible

La disponibilité

BookingService délègue cette recherche à AvailabilityService::findAvailableBoat(). Celui-ci examine les bateaux actifs correspondant au modèle demandé et vérifie, pour chacun d'eux, l'absence de réservation bloquante sur le créneau concerné. Cette vérification s'appuie notamment sur la méthode BookingRepository::hasBlockingBooking(), présentée précédemment.

```

$boats = $this->boatRepository->findActiveByBoatModel($boatModel);

foreach ($boats as $boat) {
    $hasConflict = $this->bookingRepository->hasBlockingBooking(
        $boat,
        $startAt,
        $endAt,
    );

    if (!$hasConflict) {
        return $boat;
    }
}

```

AvailabilityService - findAvailableBoat() - Recherche d'un bateau physique disponible

Création de la réservation

Une fois le bateau sélectionné, BookingService::makeBooking() construit les horaires de début et de fin de la location à partir du tarif choisi. Il crée ensuite la réservation et lui associe notamment l'utilisateur, le modèle demandé, le bateau physique qui vient d'être trouvé, le tarif, le nombre de passagers, le prix et le créneau.

```

$booking = new Booking();

$booking
    ->setUser($user)
    ->setBoatModel($boatModel)
    ->setBoat($boat)
    ->setRentalRate($rentalRate)
    ->setRentalRateLabel($rentalRate->getLabel())
    ->setPassengerCount($passengerCount)
    ->setTotalPrice($rentalRate->getPrice())
    ->setStartAt($startAt)
    ->setEndAt($endAt);

```

BookingService::makeBooking() - Création de la réservation et attribution du bateau disponible

Un paiement est également créé et associé à la réservation. La réservation et son paiement sont ensuite persistés en base de données avec Doctrine par BookingService::makeBooking(). À ce stade, la réservation existe dans l'application, mais elle reste en attente de la validation du paiement.

```

$payment = new Payment();

$payment
    ->setBooking($booking)
    ->setAmount($rentalRate->getPrice());

// Save the booking and payment.
$this->entityManager->persist($booking);
$this->entityManager->persist($payment);
$this->entityManager->flush();

return $booking;
}

```

BookingService::makeBooking() - Création du paiement et persistance de la réservation

9.2 Paiement et confirmation de la réservation

À l'issue de la création de la réservation, celle-ci possède un paiement associé encore en attente. BookingController fait alors appel à StripeService::createCheckoutSession() afin de créer une session de paiement Stripe Checkout.

```
// Creates the Stripe Checkout Session with the booking payment data.
$session = $this->stripe->checkout->sessions->create([
    'mode' => 'payment',
    'client_reference_id' => (string) $booking->getId(),

    'line_items' => [
        [
            'price_data' => [
                'currency' => 'eur',
                'unit_amount' => $payment->getAmount(),
                'product_data' => [
                    'name' => 'Location de bateau - Entre 2 Rives',
                ],
            ],
            'quantity' => 1,
        ],
    ],

    'success_url' => $this->appUrl . '/stripe/succes',
    'cancel_url' => $this->appUrl . '/stripe/annulation',
]);
```

StripeService::createCheckoutSession() - Création de la session de paiement Stripe Checkout

La session contient notamment le montant à payer ainsi que l'identifiant de la réservation. Stripe retourne ensuite une URL vers laquelle l'utilisateur est redirigé pour effectuer son paiement.

L'identifiant de la session Stripe est également enregistré dans le paiement associé à la réservation par StripeService::createCheckoutSession() :

```
// Stores the Stripe Session ID for later webhook verification.
$payment->setStripeSessionId($session->id);

$this->entityManager->flush();
```

StripeService::createCheckoutSession() - Enregistrement de l'identifiant de la session Stripe

Réception du webhook Stripe

Après le paiement, le retour de l'utilisateur sur la page de succès ne suffit pas à confirmer la réservation. La confirmation repose sur un webhook : Stripe envoie une requête HTTP POST vers la route /stripe/webhook de l'application.

La méthode StripeController::webhook() récupère l'événement et vérifie sa signature à l'aide du secret du webhook. Lorsque l'événement reçu est de type checkout.session.completed, la session est transmise à StripeService::handleCompletedCheckout() afin de poursuivre le traitement.

```

/**
 * Receives and verifies Stripe webhook events.
 */
#[Route('/stripe/webhook', name: 'app_stripe_webhook', methods: ['POST'])]
public function webhook(
    Request $request,
    StripeService $stripeService,
    #[Autowire('%env(STRIPE_WEBHOOK_SECRET)%')]
    string $webhookSecret,
): Response {
    $payload = $request->getContent();
    $signature = $request->headers->get('Stripe-Signature');

    if ($signature === null) {
        return new Response(status: Response::HTTP_BAD_REQUEST);
    }

    try {
        $event = Webhook::constructEvent(
            $payload,
            $signature,
            $webhookSecret
        );
    } catch (UnexpectedValueException | SignatureVerificationException) {
        return new Response(status: Response::HTTP_BAD_REQUEST);
    }

    if (
        $event->type === 'checkout.session.completed'
        && $event->data->object instanceof Session
    ) {
        $stripeService->handleCompletedCheckout($event->data->object);
    }
}

```

StripeController::webhook() - Réception et vérification du webhook Stripe

Confirmation de la réservation

La méthode `StripeService::handleCompletedCheckout()` reçoit la session transmise par le webhook. Elle utilise l'identifiant de la réservation enregistré dans la session Stripe pour retrouver la réservation concernée.

```

/**
 * Confirms a booking after a completed Stripe Checkout Session.
 */
public function handleCompletedCheckout(Session $session): void
{
    $bookingId = $session->client_reference_id;

    if ($bookingId === null) {
        return;
    }

    $booking = $this->bookingRepository->find($bookingId);

    if ($booking === null) {
        return;
    }
}

```

StripeService::handleCompletedCheckout() - Récupération de la réservation

Ensuite, `StripeService::handleCompletedCheckout()` récupère le paiement associé et effectue plusieurs vérifications avant de modifier les données. La méthode vérifie notamment que le paiement n'a pas déjà été traité et que l'identifiant de la session reçue correspond bien à celui enregistré lors de la création du paiement.

Lorsque les vérifications sont validées, `StripeService::handleCompletedCheckout()` enregistre l'identifiant du paiement Stripe. Le paiement passe au statut PAID et la réservation au statut CONFIRMED. Ces modifications sont ensuite enregistrées en base de données.

```
// Confirms the payment and the related booking.
$payment->setStripePaymentIntentId($paymentIntentId);
$payment->setPaymentStatus(Payment::STATUS_PAID);
$booking->setStatus(Booking::STATUS_CONFIRMED);

$this->entityManager->flush();
}
```

StripeService::handleCompletedCheckout() - Confirmation du paiement et de la réservation

9.3 Annulation et remboursement

Lorsqu'un utilisateur demande l'annulation d'une réservation, `BookingCancellationService::checkCancellation()` vérifie d'abord que celle-ci peut être annulée. La réservation doit être confirmée, posséder un paiement associé et, pour un client, la demande doit être effectuée au moins 48 heures avant le départ.

```
/**
 * Checks whether a booking can be cancelled.
 */
private function checkCancellation(
    Booking $booking,
    bool $isAdmin = false,
): void {
    if ($booking->getStatus() !== Booking::STATUS_CONFIRMED) {
        throw new \LogicException(
            'Seule une réservation confirmée peut être annulée.'
        );
    }

    if ($booking->getPayment() === null) {
        throw new \LogicException(
            'Aucun paiement n'est associé à cette réservation.'
        );
    }

    $now = new \DateTimeImmutable();
    $cancellationDeadline = $booking->getStartAt()->modify('-48 hours');

    // Allows the admin to bypass the 48-hour cancellation limit.
    if (!$isAdmin && $now > $cancellationDeadline) {
        throw new \LogicException(
            'La réservation ne peut plus être annulée moins de 48 heures avant le départ.'
        );
    }
}
```

BookingCancellationService::checkCancellation() - Vérification des conditions d'annulation

Lorsque l'annulation est autorisée, `BookingCancellationService::cancel()` fait passer la réservation au statut `CANCELLATION_PENDING` et le paiement au statut `REFUND_PENDING`. Ces états intermédiaires permettent d'indiquer que l'annulation et le remboursement ont été demandés, mais ne sont pas encore confirmés.

```
$booking->setStatus(Booking::STATUS_CANCELLATION_PENDING);
$payment->setPaymentStatus(Payment::STATUS_REFUND_PENDING);

$this->entityManager->flush();

$this->stripeService->createRefund($payment);
}
```

BookingCancellationService::cancel() - Passage de la réservation et du paiement aux statuts d'attente

`BookingCancellationService::cancel()` demande ensuite le remboursement à Stripe par l'intermédiaire de `StripeService::createRefund()`.

Demande de remboursement à Stripe

La méthode `StripeService::createRefund()` vérifie que le paiement est bien en attente de remboursement et qu'un identifiant de paiement Stripe lui est associé. Elle transmet ensuite à Stripe une demande de remboursement à partir de cet identifiant.

```
$paymentIntentId = $payment->getStripePaymentIntentId();

if ($paymentIntentId === null) {
    throw new \LogicException(
        'Aucun identifiant de paiement Stripe n'est associé à ce paiement.'
    );
}

try {
    // Requests the refund using the original Stripe Payment Intent.
    $this->stripe->refunds->create([
        'payment_intent' => $paymentIntentId,
        'reason' => 'requested_by_customer',
        'metadata' => [
            'booking_id' => (string) $payment->getBooking()->getId(),
        ],
    ]);
}
```

StripeService::createRefund() - Création de la demande de remboursement Stripe

L'identifiant de la réservation est également ajouté aux métadonnées de la demande de remboursement. Lorsque Stripe transmettra le résultat de l'opération au webhook, cet identifiant permettra à `StripeService::handleCreatedRefund()` de retrouver la réservation concernée.

Lorsque Stripe envoie l'événement `refund.created`, `StripeController::webhook()` le reçoit et confie son traitement à `StripeService::handleCreatedRefund()`.

```
if (
    $event->type === 'refund.created'
    && $event->data->object instanceof Refund
) {
    $stripeService->handleCreatedRefund($event->data->object);
}
```

StripeController::webhook() - Réception de l'événement de remboursement

Après avoir retrouvé la réservation concernée et vérifié que l'annulation et le remboursement sont bien en attente, `StripeService::handleCreatedRefund()` finalise le traitement. Le paiement passe au statut `REFUNDED` et la réservation au statut `CANCELLED`. Ces modifications sont ensuite enregistrées en base de données.

```
// Retrieves the booking ID previously stored in Stripe metadata.
$bookingId = $refund->metadata->booking_id ?? null;

if ($bookingId === null) {
    return;
}

$booking = $this->bookingRepository->find($bookingId);

if ($booking === null) {
    return;
}

if ($booking->getStatus() !== Booking::STATUS_CANCELLATION_PENDING) {
    return;
}

$payment = $booking->getPayment();

if ($payment === null) {
    return;
}

if ($payment->getPaymentStatus() !== Payment::STATUS_REFUND_PENDING) {
    return;
}

// Finalizes the refund and cancellation after Stripe confirmation.
$payment->setPaymentStatus(Payment::STATUS_REFUNDED);
$booking->setStatus(Booking::STATUS_CANCELLED);

$this->entityManager->flush();
}
```

StripeService::handleCreatedRefund() - Confirmation du remboursement et finalisation de l'annulation

10. Administration et gestion des données

10.1 Tableau de bord et accès à l'administration

L'application dispose d'un espace d'administration destiné à la gestion de l'activité d'Entre 2 Rives. Son tableau de bord constitue le point d'entrée de cet espace. Il permet à l'administrateur de consulter rapidement les réservations prévues pour la journée et d'accéder aux différentes interfaces de gestion.

Tableau de bord

Bienvenue dans l'espace d'administration d'Entre 2 Rives.

Réservations du jour

Client	Horaire	Modèle	Formule	Statut
jessie Gautherot	14 h 00	Escapade	Après-midi	Confirmée
jessie Gautherot	16 h 00	Évasion	Fin d'après-midi	Confirmée
jessie Gautherot	16 h 00	Grand Large	Fin d'après-midi	Confirmée

Gestion

Réservations

Consulter et gérer les réservations.

Accéder

Bateaux

Gérer la disponibilité des bateaux physiques.

Accéder

Modèles

Gérer les modèles proposés à la location.

Accéder

Tarifs de location

Gérer les formules et leurs tarifs.

Accéder

Clients

Consulter les informations des clients.

Accéder

Tableau de bord - Vue d'ensemble de l'espace d'administration

L'accès au tableau de bord est défini par la route `/admin`. Le contrôleur correspondant est réservé aux utilisateurs disposant du rôle `ROLE_ADMIN`.

```
#[IsGranted('ROLE_ADMIN')]  
class DashboardController extends AbstractController  
{
```

AdminController - Contrôle de l'accès au tableau de bord

Lors de l'affichage du tableau de bord, le contrôleur utilise `BookingRepository::findTodayBookings()` pour récupérer les réservations prévues pour la journée. Les données obtenues sont ensuite transmises au template Twig afin de générer le tableau des réservations du jour.

Celui-ci présente les principales informations utiles au gestionnaire : le client, l'horaire de la location, le modèle de bateau, le tarif choisi et le statut de la réservation.

Le tableau de bord centralise également les accès aux différentes fonctions de l'administration : gestion des réservations, des bateaux physiques, des modèles, des tarifs, ainsi que consultation des clients.

L'administration ne propose cependant pas les mêmes opérations pour toutes ces ressources. Les actions disponibles ont été adaptées aux besoins de gestion de l'application. Cette organisation est présentée dans la partie suivante à travers les différentes opérations proposées et l'exemple de la gestion des tarifs.

10.2 Gestion des données et opérations d'administration

Les différentes interfaces d'administration permettent de gérer les données nécessaires au fonctionnement d'Entre 2 Rives. Les opérations proposées ne sont pas identiques pour toutes les ressources : elles ont été définies en fonction des besoins de gestion de l'application.

Les réservations peuvent être consultées et annulées, les bateaux physiques peuvent être rendus disponibles ou indisponibles, les informations des modèles peuvent être modifiées et les données des clients peuvent être consultées.

La gestion des tarifs constitue un exemple plus complet. L'administrateur peut consulter les tarifs existants, en créer de nouveaux, les modifier et les rendre disponibles ou indisponibles.

[Retour au tableau de bord](#)

Tarifs de location

Gérez les formules proposées.

[Créer une formule](#)

Escapade

Formule	Durée	Horaires	Prix	Disponibilité	Actions
Petit Matin	2 h	09 h 00 — 11 h 00	50,00 €	Disponible	Modifier Rendre indisponible
Midi	2 h	11 h 00 — 13 h 00	50,00 €	Disponible	Modifier Rendre indisponible
Début d'après-midi	2 h	14 h 00 — 16 h 00	50,00 €	Disponible	Modifier Rendre indisponible
Fin d'après-midi	2 h	16 h 00 — 18 h 00	50,00 €	Disponible	Modifier Rendre indisponible
Matinée	4 h	09 h 00 — 13 h 00	90,00 €	Disponible	Modifier Rendre indisponible
Après-midi	4 h	14 h 00 — 18 h 00	90,00 €	Disponible	Modifier Rendre indisponible
Journée	9 h	09 h 00 — 18 h 00	150,00 €	Disponible	Modifier Rendre indisponible

Évasion

Formule	Durée	Horaires	Prix	Disponibilité	Actions
Petit Matin	2 h	09 h 00 — 11 h 00	80,00 €	Disponible	Modifier Rendre indisponible
Midi	2 h	11 h 00 — 13 h 00	80,00 €	Disponible	Modifier Rendre indisponible
Début d'après-midi	2 h	14 h 00 — 16 h 00	80,00 €	Disponible	Modifier Rendre indisponible

Administration des RentalRate - Gestion des tarifs

Ces opérations sont regroupées dans `RentalRateAdminController`. Le contrôleur définit une route commune `/admin/tarifs`, complétée par différentes routes correspondant aux actions disponibles : afficher la liste des tarifs, en créer un avec `/creer`, en modifier un avec `/ {id} /modifier` et changer sa disponibilité avec `/ {id} /disponibilite`.

L'action `new()` utilise `RentalRateFormType` pour construire le formulaire associé à une nouvelle instance de `RentalRate`. Après la soumission, `handleRequest()` récupère les données envoyées et le contrôleur vérifie que le formulaire est soumis et valide. Le nouveau tarif est alors persisté puis enregistré en base de données avec Doctrine.

La modification repose sur le même `RentalRateFormType`, cette fois associé à un tarif existant. Lorsque les données modifiées sont valides, `flush()` enregistre les changements en base de données.

Un tarif n'est pas supprimé lorsqu'il ne doit plus être proposé à la réservation. L'action `toggleAvailability()` permet de modifier son état `isActive`.

Cette action utilise uniquement la méthode HTTP POST et vérifie également un jeton CSRF avant d'effectuer la modification.

Ce fonctionnement permet de conserver les tarifs enregistrés tout en contrôlant ceux qui peuvent être proposés aux clients. Le même principe d'activation ou de désactivation est également utilisé pour les bateaux physiques.

11. Sécurité et protection des données

11.1 Authentification et contrôle des accès

L'application distingue les fonctionnalités accessibles publiquement de celles réservées aux utilisateurs authentifiés ou aux administrateurs. Cette gestion repose sur le composant Security de Symfony, les rôles associés aux utilisateurs et des contrôles complémentaires liés aux règles métier.

Création et sécurisation du compte utilisateur

Lors de l'inscription, les données saisies sont contrôlées à l'aide des contraintes de validation Symfony. L'adresse e-mail doit notamment être valide et unique. Le mot de passe doit respecter les règles définies dans `RegistrationFormType` et sa confirmation est vérifiée avant la création du compte.

Le champ `plainPassword` est défini avec l'option `mapped: false` : il n'est donc pas associé à une propriété persistée de l'entité `User`. Le mot de passe en clair n'est utilisé que pendant le traitement du formulaire. Les règles de complexité sont vérifiées sur cette valeur avant que le mot de passe ne soit haché puis enregistré.

Avant la persistance de l'utilisateur, le mot de passe est traité avec `UserPasswordHasherInterface`. Seul le mot de passe haché est ensuite affecté à l'entité `User`.

Le même traitement prépare l'activation du compte : celui-ci est créé avec la propriété `isActive` à `false` et un jeton d'activation lui est associé.

```
if ($form->isSubmitted() && $form->isValid()) {  
    // Get the plain password from the form.  
    $plainPassword = $form->get('plainPassword')->getData();  
  
    // Hash the password before saving the user.  
    $hashedPassword = $passwordHasher->hashPassword(  
        $user,  
        $plainPassword  
    );  
  
    $user->setPassword($hashedPassword);  
  
    // Prepare account activation.  
    $user->setActive(false);  
    $userActivationService->generateToken($user);  
}
```

RegistrationController - Hachage du mot de passe et préparation de l'activation du compte.

Activation du compte

Un jeton d'activation est construit à partir de 32 octets générés aléatoirement avec `random_bytes()`, puis convertis en chaîne hexadécimale. Il est associé au compte pendant la phase d'activation et envoyé à l'utilisateur dans un lien par e-mail. Lorsqu'il utilise le lien correspondant, `UserActivationService` recherche le compte à partir de ce jeton et l'active.

Une fois le lien utilisé, le compte passe à l'état actif et le jeton est remplacé par `null`. Le même lien ne permet donc plus de retrouver l'utilisateur lors d'une nouvelle tentative d'activation.

Authentification

Le `UserAuthenticator` construit un `Passport` contenant notamment un `UserBadge`, utilisé pour identifier l'utilisateur à partir de son adresse e-mail, et des `PasswordCredentials`, qui permettent à Symfony de vérifier son mot de passe.

Le formulaire de connexion est également protégé par un jeton CSRF.

L'activation du compte reste distincte de l'authentification : un utilisateur peut être authentifié alors que son compte n'est pas encore actif.

Contrôle des accès par les rôles

Une fois authentifié, les rôles permettent de contrôler l'accès aux différentes parties de l'application.

Les utilisateurs authentifiés disposent du rôle `ROLE_USER`, tandis que les interfaces d'administration sont réservées à `ROLE_ADMIN`.

Ces restrictions sont placées directement sur les contrôleurs concernés avec l'attribut `#[IsGranted]`. Par exemple, l'ensemble du contrôleur de réservation nécessite le rôle `ROLE_USER` :

```
#[IsGranted('ROLE_USER')]
class BookingController extends AbstractController
```

BookingController - Restriction de l'accès aux utilisateurs disposant du rôle `ROLE_USER`

Contrôle complémentaire par la logique métier

Le contrôle d'accès par les rôles est complété par les règles métier de l'application. Ainsi, un utilisateur authentifié peut accéder à l'interface de réservation, mais son compte doit être actif pour créer effectivement une réservation.

Cette condition est vérifiée côté serveur par `BookingService` lors de la soumission du formulaire. Si `isActive` vaut `false`, le traitement est interrompu et l'utilisateur est informé qu'il doit d'abord activer son compte.

11.2 Sécurisation des actions et des données

Protection des actions sensibles

Les actions qui modifient les données de l'application font l'objet de contrôles spécifiques côté serveur. L'annulation d'une réservation permet d'illustrer plusieurs de ces mécanismes.

La route correspondante accepte uniquement la méthode HTTP `POST`. Avant de poursuivre le traitement, le contrôleur vérifie également que la réservation appartient bien à l'utilisateur connecté. Un utilisateur ne peut donc pas annuler la réservation d'un autre client en utilisant directement son identifiant dans l'URL.

L'action est également protégée contre les attaques CSRF. Le formulaire génère un jeton associé à la réservation à partir de son identifiant :

```

<input
  type="hidden"
  name="_token"
  value="{{ csrf_token('cancel-booking-' ~ booking.id) }}"
>

```

account/index.html.twig - Génération du jeton CSRF pour l'annulation d'une réservation

Lors de la soumission, ce jeton est récupéré puis vérifié côté serveur avec `isCsrfTokenValid()`. S'il est invalide, une exception d'accès est déclenchée et le traitement est interrompu.

```

// Checks that the booking belongs to the connected user.
if ($booking->getUser() !== $this->getUser()) {
    throw $this->createAccessDeniedException();
}

// Checks the CSRF token before processing the cancellation.
if (!$this->isCsrfTokenValid(
    'cancel-booking-' . $booking->getId(),
    $request->request->get('_token'),
)) {
    throw $this->createAccessDeniedException(
        'Jeton CSRF invalide.',
    );
}

```

BookingController – cancel - Contrôle du propriétaire et vérification du jeton CSRF avant l'annulation

Ce n'est qu'après ces contrôles que le contrôleur transmet la réservation à `BookingCancellationService`, chargé d'appliquer les règles métier propres à l'annulation.

Sécurisation de l'accès aux données

L'accès à la base de données est réalisé avec Doctrine ORM et les repositories. Les requêtes personnalisées utilisent le `QueryBuilder` et transmettent les valeurs sous forme de paramètres, plutôt que de les intégrer directement dans les requêtes.

Ce fonctionnement a notamment été présenté au chapitre 8 avec `BookingRepository::hasBlockingBooking()`, utilisé pour rechercher les réservations qui se chevauchent. Les valeurs nécessaires à cette recherche sont transmises à la requête sous forme de paramètres.

L'utilisation de requêtes paramétrées évite de concaténer directement des valeurs dans les requêtes SQL et participe ainsi à la protection de l'application contre les injections SQL.

11.3 Protection des données sensibles et personnelles

Protection des secrets techniques

L'application utilise plusieurs informations sensibles nécessaires à la connexion à la base de données et aux services externes, notamment Stripe et le service d'envoi d'e-mails. Ces valeurs ne sont pas écrites directement dans le code source : elles sont définies à l'aide de variables d'environnement.

Dans l'environnement de développement, les valeurs propres à l'installation sont renseignées dans le fichier `.env.local`. C'est notamment le cas de la clé secrète Stripe, du secret utilisé pour les webhooks ou encore des informations de connexion à la base de données et au serveur de messagerie.

Le fichier `.env.local` est exclu du suivi Git par le fichier `.gitignore`.

Les valeurs sensibles propres à l'environnement local ne sont ainsi pas destinées à être enregistrées dans le dépôt avec le code source.

Le secret du webhook Stripe est également utilisé pour vérifier l'origine des événements reçus par l'application. Il est injecté dans le contrôleur à partir de la variable d'environnement `STRIPE_WEBHOOK_SECRET`.

Lorsqu'une requête arrive sur l'endpoint `/stripe/webhook`, le contrôleur récupère son contenu ainsi que l'en-tête `Stripe-Signature`. La méthode `Webhook::constructEvent()` utilise ensuite ces informations avec le secret du webhook pour vérifier la signature de l'événement.

Si la signature est absente ou si sa vérification échoue, le contrôleur retourne une réponse HTTP 400 et l'événement n'est pas traité. Les traitements liés au paiement ou au remboursement ne sont exécutés qu'après cette vérification.

```
$payload = $request->getContent();
$signature = $request->headers->get('Stripe-Signature');

if ($signature === null) {
    return new Response(status: Response::HTTP_BAD_REQUEST);
}

try {
    $event = Webhook::constructEvent(
        $payload,
        $signature,
        $webhookSecret
    );
} catch (UnexpectedValueException | SignatureVerificationException) {
    return new Response(status: Response::HTTP_BAD_REQUEST);
}
```

StripeController - Vérification de la signature d'un événement Stripe avant son traitement.

Protection des données personnelles

L'application traite plusieurs données personnelles nécessaires à son fonctionnement, notamment le nom, le prénom, l'adresse e-mail et le numéro de téléphone lors de la création d'un compte. Les informations liées aux réservations sont également associées à l'utilisateur afin de permettre leur gestion et leur consultation depuis l'espace personnel.

Les données bancaires constituent un cas particulier : le paiement est réalisé sur l'interface de Stripe et les informations bancaires ne sont pas enregistrées par Entre 2 Rives. Dans le cadre du projet, Stripe est utilisé uniquement en mode test.

Une politique de confidentialité accessible sur le site informe l'utilisateur sur les données traitées et leur utilisation. Elle présente également leur conservation, les services externes pouvant intervenir, les mesures de sécurité mises en œuvre et l'utilisation de cookies nécessaires au fonctionnement de l'application, notamment pour la gestion de la session. Aucun dispositif de suivi publicitaire ou commercial n'est intégré au projet.

Elle précise enfin les droits de l'utilisateur concernant ses données personnelles, notamment les droits d'accès, de rectification et d'effacement ainsi que les modalités permettant d'effectuer une demande.

12. Référencement

Pour favoriser le référencement des pages publiques, plusieurs bonnes pratiques sont intégrées directement dans la structure des interfaces.

Les templates Twig génèrent un HTML structuré et sémantique. Les pages utilisent notamment des éléments adaptés à leur rôle, comme les différents niveaux de titres h1, h2 et h3, les éléments de navigation ou les sections de contenu. Cette organisation facilite la compréhension de la structure des pages, aussi bien pour les utilisateurs que pour les moteurs de recherche.

Chaque page définit également un titre adapté à son contenu à l'aide du bloc title de Twig.

Les URLs publiques sont également conçues pour rester lisibles. Les fiches des modèles de bateaux utilisent notamment un slug afin d'obtenir une URL descriptive plutôt qu'un simple identifiant.

Enfin, un fichier robots.txt placé dans le répertoire public indique aux robots d'indexation quelles parties du site peuvent être explorées. Les pages publiques restent accessibles, tandis que les espaces privés et certaines routes techniques sont exclus de l'exploration.

13. Tests et validation

13.1 Stratégie et environnement de test

Des tests automatisés ont été mis en place afin de vérifier les principales fonctionnalités de l'application. Ils sont exécutés avec PHPUnit et s'appuient sur les outils de test fournis par Symfony.

Les tests nécessitant l'accès aux données utilisent l'environnement de test de Symfony et une base MySQL dédiée, `entre_deux_rives_test`. Cette séparation permet d'exécuter les scénarios sans modifier les données utilisées pendant le développement.

Le type de test est choisi en fonction du comportement à vérifier : une logique isolée peut être testée unitairement, tandis que les fonctionnalités faisant intervenir plusieurs composants de l'application sont vérifiées à travers des tests d'intégration ou fonctionnels.

Certaines validations de l'entité User font par exemple l'objet de tests unitaires.

`testInvalidEmail()` crée un utilisateur avec une adresse e-mail invalide puis utilise le validateur Symfony pour vérifier qu'une erreur de validation est bien générée.

```
/**
 * Check that an invalid email address is rejected.
 */
public function testInvalidEmail(): void
{
    $user = $this->createValidUser();
    $user->setEmail('email-invalid');

    $errors = $this->createValidator()
        ->validateProperty($user, 'email');

    self::assertGreaterThan(0, count($errors));
}
```

UserTest::testInvalidEmail() - Validation d'une adresse e-mail invalide

13.2 Test de la disponibilité d'un bateau

Lors de la création d'une réservation, l'application doit attribuer un bateau physique disponible correspondant au modèle choisi par le client. Cette recherche est assurée par `AvailabilityService::findAvailableBoat()`.

Cette règle fait l'objet d'un test d'intégration dans `AvailableBoatTest`. Un premier scénario vérifie qu'un bateau est bien retourné lorsqu'il est disponible pour le modèle, la date et un tarif demandés.

Un second scénario reproduit le cas où tous les bateaux actifs du modèle sont déjà réservés sur le même créneau. Des réservations sont créées dans la base de test pour chacun de ces bateaux, puis `findAvailableBoat()` est appelée. Le test vérifie alors qu'aucun bateau n'est retourné.

```
// Books every active boat for the same date and time.
foreach ($boats as $boat) {
    $booking = $this->createBooking([
        $user,
        $boat,
        $rentalRate,
        $startAt,
        $endAt,
    ]);

    $entityManager->persist($booking);
    $bookings[] = $booking;
}

$entityManager->flush();

$availableBoat = $availabilityService->findAvailableBoat(
    $boatModel,
    $rentalRate,
    $date,
);

$this->assertNull($availableBoat);
```

AvailableBoatTest::testNoBoatAvailableWhenAllAreBooked() - Vérification de l'absence de bateau disponible lorsque tous sont réservés

AvailableBoatTest utilise KernelTestCase afin de démarrer l'application Symfony et d'accéder au véritable service de disponibilité, aux repositories Doctrine et à la base de données de test. Le scénario vérifie ainsi le fonctionnement de ces différents composants ensemble.

13.3 Test fonctionnel du parcours de réservation

Le parcours de réservation fait également l'objet de tests fonctionnels avec WebTestCase. Contrairement au test d'intégration précédent, ce type de test permet de simuler les requêtes HTTP effectuées par un utilisateur dans l'application.

BookingControllerTest::testCreateBooking() connecte un utilisateur de test, récupère un jeton CSRF depuis le formulaire de réservation puis envoie une requête POST contenant des données valides. Le test vérifie ensuite que la réservation et le paiement associé ont bien été créés et que l'application redirige l'utilisateur vers Stripe Checkout.

L'appel à Stripe n'est pas réellement effectué pendant le test. StripeService est simulé afin de retourner une URL de paiement fictive. Le parcours interne de l'application peut ainsi être vérifié sans dépendre d'un service externe.

```

// Submits a valid booking form.
$client->request(
    'POST',
    '/reservation',
    [
        '_token' => $token,
        'date' => $date->format('Y-m-d'),
        'passengerCount' => 2,
        'boatModel' => $boatModel->getId(),
        'rentalRate' => $rentalRate->getId(),
        'acceptTerms' => '1',
    ],
);

$this->assertResponseRedirects(
    'https://checkout.stripe.test/session',
);

[...]

// Checks that the booking was created.
$booking = $entityManager
    ->getRepository(Booking::class)
    ->findOneBy([
        [...]
    ]);

$this->assertNotNull($booking);

// Checks that the related payment was also created.
$payment = $booking->getPayment();

$this->assertNotNull($payment);

```

BookingControllerTest::testCreateBooking() - Test fonctionnel de la création d'une réservation

13.4 Exécution et résultats des tests

Les tests automatisés sont exécutés avec PHPUnit. Leur exécution permet de vérifier les différents scénarios couverts et d'identifier rapidement un éventuel échec.

php bin/phpunit

```

OK, but there were issues!
Tests: 9, Assertions: 27, Deprecations: 4.
PS C:\Users\User\Desktop\Entre 2 rives>

```

Résultats des tests dans le terminal

L'exécution présentée ci-dessus confirme que l'ensemble des tests exécutés est validé. Elle fait également apparaître quatre avertissements de dépréciation liés à une syntaxe encore prise en charge par les versions actuelles des composants utilisés, mais amenée à évoluer. Ces avertissements n'empêchent pas l'exécution ni la validation des tests.

14. Déploiement et documentation

14.1 Documentation du projet

Le projet est accompagné d'un fichier README.md qui regroupe les informations nécessaires à son installation et à sa prise en main.

Il présente les prérequis techniques, le clonage et l'installation du projet, la configuration des environnements de développement et de test ainsi que la création des bases de données à l'aide des migrations et des fixtures.

Le README décrit également les principales étapes permettant de tester l'application : utilisation des comptes de démonstration, parcours de réservation, paiement avec Stripe en mode test et exécution des tests automatisés avec PHPUnit.

Enfin, il présente l'organisation générale des principaux répertoires du projet afin de faciliter la compréhension de sa structure.

```
README.md 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1041 1042 1043 1044 1045 1046 1047 1048 1049 1050 1051 1052 1053 1054 1055 1056 1057 1058 1059 1060 1061 1062 1063 1064 1065 1066 1067 1068 1069 1070 1071 1072 1073 1074 1075 1076 1077 1078 1079 1080 1081 1082 1083 1084 1085 1086 1087 1088 1089 1090 1091 1092 1093 1094 1095 1096 1097 1098 1099 1100 1101 1102 1103 1104 1105 1106 1107 1108 1109 1110 1111 1112 1113 1114 1115 1116 1117 1118 1119 1120 1121 1122 1123 1124 1125 1126 1127 1128 1129 1130 1131 1132 1133 1134 1135 1136 1137 1138 1139 1140 1141 1142 1143 1144 1145 1146 1147 1148 1149 1150 1151 1152 1153 1154 1155 1156 1157 1158 1159 1160 1161 1162 1163 1164 1165 1166 1167 1168 1169 1170 1171 1172 1173 1174 1175 1176 1177 1178 1179 1180 1181 1182 1183 1184 1185 1186 1187 1188 1189 1190 1191 1192 1193 1194 1195 1196 1197 1198 1199 1200 1201 1202 1203 1204 1205 1206 1207 1208 1209 1210 1211 1212 1213 1214 1215 1216 1217 1218 1219 1220 1221 1222 1223 1224 1225 1226 1227 1228 1229 1230 1231 1232 1233 1234 1235 1236 1237 1238 1239 1240 1241 1242 1243 1244 1245 1246 1247 1248 1249 1250 1251 1252 1253 1254 1255 1256 1257 1258 1259 1260 1261 1262 1263 1264 1265 1266 1267 1268 1269 1270 1271 1272 1273 1274 1275 1276 1277 1278 1279 1280 1281 1282 1283 1284 1285 1286 1287 1288 1289 1290 1291 1292 1293 1294 1295 1296 1297 1298 1299 1300 1301 1302 1303 1304 1305 1306 1307 1308 1309 1310 1311 1312 1313 1314 1315 1316 1317 1318 1319 1320 1321 1322 1323 1324 1325 1326 1327 1328 1329 1330 1331 1332 1333 1334 1335 1336 1337 1338 1339 1340 1341 1342 1343 1344 1345 1346 1347 1348 1349 1350 1351 1352 1353 1354 1355 1356 1357 1358 1359 1360 1361 1362 1363 1364 1365 1366 1367 1368 1369 1370 1371 1372 1373 1374 1375 1376 1377 1378 1379 1380 1381 1382 1383 1384 1385 1386 1387 1388 1389 1390 1391 1392 1393 1394 1395 1396 1397 1398 1399 1400 1401 1402 1403 1404 1405 1406 1407 1408 1409 1410 1411 1412 1413 1414 1415 1416 1417 1418 1419 1420 1421 1422 1423 1424 1425 1426 1427 1428 1429 1430 1431 1432 1433 1434 1435 1436 1437 1438 1439 1440 1441 1442 1443 1444 1445 1446 1447 1448 1449 1450 1451 1452 1453 1454 1455 1456 1457 1458 1459 1460 1461 1462 1463 1464 1465 1466 1467 1468 1469 1470 1471 1472 1473 1474 1475 1476 1477 1478 1479 1480 1481 1482 1483 1484 1485 1486 1487 1488 1489 1490 1491 1492 1493 1494 1495 1496 1497 1498 1499 1500 1501 1502 1503 1504 1505 1506 1507 1508 1509 1510 1511 1512 1513 1514 1515 1516 1517 1518 1519 1520 1521 1522 1523 1524 1525 1526 1527 1528 1529 1530 1531 1532 1533 1534 1535 1536 1537 1538 1539 1540 1541 1542 1543 1544 1545 1546 1547 1548 1549 1550 1551 1552 1553 1554 1555 1556 1557 1558 1559 1560 1561 1562 1563 1564 1565 1566 1567 1568 1569 1570 1571 1572 1573 1574 1575 1576 1577 1578 1579 1580 1581 1582 1583 1584 1585 1586 1587 1588 1589 1590 1591 1592 1593 1594 1595 1596 1597 1598 1599 1600 1601 1602 1603 1604 1605 1606 1607 1608 1609 1610 1611 1612 1613 1614 1615 1616 1617 1618 1619 1620 1621 1622 1623 1624 1625 1626 1627 1628 1629 1630 1631 1632 1633 1634 1635 1636 1637 1638 1639 1640 1641 1642 1643 1644 1645 1646 1647 1648 1649 1650 1651 1652 1653 1654 1655 1656 1657 1658 1659 1660 1661 1662 1663 1664 1665 1666 1667 1668 1669 1670 1671 1672 1673 1674 1675 1676 1677 1678 1679 1680 1681 1682 1683 1684 1685 1686 1687 1688 1689 1690 1691 1692 1693 1694 1695 1696 1697 1698 1699 1700 1701 1702 1703 1704 1705 1706 1707 1708 1709 1710 1711 1712 1713 1714 1715 1716 1717 1718 1719 1720 1721 1722 1723 1724 1725 1726 1727 1728 1729 1730 1731 1732 1733 1734 1735 1736 1737 1738 1739 1740 1741 1742 1743 1744 1745 1746 1747 1748 1749 1750 1751 1752 1753 1754 1755 1756 1757 1758 1759 1760 1761 1762 1763 1764 1765 1766 1767 1768 1769 1770 1771 1772 1773 1774 1775 1776 1777 1778 1779 1780 1781 1782 1783 1784 1785 1786 1787 1788 1789 1790 1791 1792 1793 1794 1795 1796 1797 1798 1799 1800 1801 1802 1803 1804 1805 1806 1807 1808 1809 1810 1811 1812 1813 1814 1815 1816 1817 1818 1819 1820 1821 1822 1823 1824 1825 1826 1827 1828 1829 1830 1831 1832 1833 1834 1835 1836 1837 1838 1839 1840 1841 1842 1843 1844 1845 1846 1847 1848 1849 1850 1851 1852 1853 1854 1855 1856 1857 1858 1859 1860 1861 1862 1863 1864 1865 1866 1867 1868 1869 1870 1871 1872 1873 1874 1875 1876 1877 1878 1879 1880 1881 1882 1883 1884 1885 1886 1887 1888 1889 1890 1891 1892 1893 1894 1895 1896 1897 1898 1899 1900 1901 1902 1903 1904 1905 1906 1907 1908 1909 1910 1911 1912 1913 1914 1915 1916 1917 1918 1919 1920 1921 1922 1923 1924 1925 1926 1927 1928 1929 1930 1931 1932 1933 1934 1935 1936 1937 1938 1939 1940 1941 1942 1943 1944 1945 1946 1947 1948 1949 1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033 2034 2035 2036 2037 2038 2039 2040 2041 2042 2043 2044 2045 2046 2047 2048 2049 2050 2051 2052 2053 2054 2055 2056 2057 2058 2059 2060 2061 2062 2063 2064 2065 2066 2067 2068 2069 2070 2071 2072 2073 2074 2075 2076 2077 2078 2079 2080 2081 2082 2083 2084 2085 2086 2087 2088 2089 2090 2091 2092 2093 2094 2095 2096 2097 2098 2099 2100 2101 2102 2103 2104 2105 2106 2107 2108 2109 2110 2111 2112 2113 2114 2115 2116 2117 2118 2119 2120 2121 2122 2123 2124 2125 2126 2127 2128 2129 2130 2131 2132 2133 2134 2135 2136 2137 2138 2139 2140 2141 2142 2143 2144 2145 2146 2147 2148 2149 2150 2151 2152 2153 2154 2155 2156 2157 2158 2159 2160 2161 2162 2163 2164 2165 2166 2167 2168 2169 2170 2171 2172 2173 2174 2175 2176 2177 2178 2179 2180 2181 2182 2183 2184 2185 2186 2187 2188 2189 2190 2191 2192 2193 2194 2195 2196 2197 2198 2199 2200 2201 2202 2203 2204 2205 2206 2207 2208 2209 2210 2211 2212 2213 2214 2215 2216 2217 2218 2219 2220 2221 2222 2223 2224 2225 2226 2227 2228 2229 2230 2231 2232 2233 2234 2235 2236 2237 2238 2239 2240 2241 2242 2243 2244 2245 2246 2247 2248 2249 2250 2251 2252 2253 2254 2255 2256 2257 2258 2259 2260 2261 2262 2263 2264 2265 2266 2267 2268 2269 2270 2271 2272 2273 2274 2275 2276 2277 2278 2279 2280 2281 2282 2283 2284 2285 2286 2287 2288 2289 2290 2291 2292 2293 2294 2295 2296 2297 2298 2299 2300 2301 2302 2303 2304 2305 2306 2307 2308 2309 2310 2311 2312 2313 2314 2315 2316 2317 2318 2319 2320 2321 2322 2323 2324 2325 2326 2327 2328 2329 2330 2331 2332 2333 2334 2335 2336 2337 2338 2339 2340 2341 2342 2343 2344 2345 2346 2347 2348 2349 2350 2351 2352 2353 2354 2355 2356 2357 2358 2359 2360 2361 2362 2363 2364 2365 2366 2367 2368 2369 2370 2371 2372 2373 2374 2375 2376 2377 2378 2379 2380 2381 2382 2383 2384 2385 2386 2387 2388 2389 2390 2391 2392 2393 2394 2395 2396 2397 2398 2399 2400 2401 2402 2403 2404 2405 2406 2407 2408 2409 2410 2411 2412 2413 2414 2415 2416 2417 2418 2419 2420 2421 2422 2423 2424 2425 2426 2427 2428 2429 2430 2431 2432 2433 2434 2435 2436 2437 2438 2439 2440 2441 2442 2443 2444 2445 2446 2447 2448 2449 2450 2451 2452 2453 2454 2455 2456 2457 2458 2459 2460 2461 2462 2463 2464 2465 2466 2467 2468 2469 2470 2471 2472 2473 2474 2475 2476 2477 2478 2479 2480 2481 2482 2483 2484 2485 2486 2487 2488 2489 2490 2491 2492 2493 2494 2495 2496 2497 2498 2499 2500 2501 2502 2503 2504 2505 2506 2507 2508 2509 2510 2511 2512 2513 2514 2515 2516 2517 2518 2519 2520 2521 2522 2523 2524 2525 2526 2527 2528 2529 2530 2531 2532 2533 2534 2535 2536 2537 2538 2539 2540 2541 2542 2543 2544 2545 2546 2547 2548 2549 2550 2551 2552 2553 2554 2555 2556 2557 2558 2559 2560 2561 2562 2563 2564 2565 2566 2567 2568 2569 2570 2571 2572 2573 2574 2575 2576 2577 2578 2579 2580 2581 2582 2583 2584 2585 2586 2587 2588 2589 2590 2591 2592 2593 2594 2595 2596 2597 2598 2599 2600 2601 26
```

14.2 Préparation au déploiement

L'application est actuellement développée et testée en environnement local. Son déploiement est prévu après la finalisation de ce dossier de projet.

La configuration de l'application a été organisée afin de séparer du code source les paramètres propres à chaque environnement. Les informations sensibles, comme les accès à la base de données, les clés Stripe ou la configuration de l'envoi d'e-mails, sont ainsi gérées à l'aide de variables d'environnement.

Le passage en production nécessitera notamment de configurer la base de données et les variables d'environnement sur le serveur, d'exécuter les migrations Doctrine et d'adapter la configuration des services externes, en particulier Stripe et l'envoi d'e-mails.

La solution d'hébergement envisagée est **Alwaysdata**, qui permet l'hébergement d'une application PHP ainsi que l'utilisation d'une base de données MySQL.

15. Difficultés rencontrées

15.1 Compréhension et mise en place des webhooks Stripe en environnement local

L'une des principales difficultés rencontrées pendant le développement a concerné la mise en place des remboursements et des webhooks Stripe dans mon environnement local.

Dès la conception du projet, je souhaitais que l'application permette non seulement de payer une réservation en ligne en mode test, mais également de rembourser un paiement lors d'une annulation et de s'appuyer sur des webhooks pour recevoir les confirmations de Stripe.

J'ai choisi de mettre en place un webhook afin que la confirmation du paiement ne dépende pas uniquement du retour de l'utilisateur sur le site, mais d'une confirmation transmise directement par Stripe. L'utilisation de Stripe en mode test et la création d'un paiement ne constituaient pas une nouveauté. En revanche, je n'avais encore jamais développé de fonctionnalité de remboursement ni mis en place la réception d'événements Stripe par webhook.

La compréhension de cet échange a constitué une première difficulté. Je devais notamment comprendre que mon application ne venait pas interroger Stripe pour savoir si le paiement avait réussi : c'est Stripe qui, lorsqu'un événement se produit, envoie une nouvelle requête HTTP vers une route prévue à cet effet dans mon application.

À l'aide de la documentation Stripe, j'ai progressivement compris le parcours de cette information. Après le paiement, Stripe génère un événement, par exemple `checkout.session.completed`, puis l'envoie par une requête POST vers `/stripe/webhook`. Mon application reçoit cette requête, vérifie sa signature pour s'assurer qu'elle provient bien de Stripe, puis traite l'événement reçu afin de mettre à jour le paiement et la réservation concernés.

Le remboursement m'a également amenée à comprendre un échange dans l'autre sens. Lorsqu'une réservation doit être annulée, mon application envoie d'abord une demande de remboursement à Stripe. Elle ne considère cependant pas immédiatement le remboursement comme terminé : elle attend à son tour l'événement envoyé par Stripe confirmant le résultat de l'opération. Des statuts intermédiaires permettent de représenter cette attente dans l'application.

Une fois ce principe compris, une seconde difficulté est apparue pour le reproduire dans mon environnement de développement : l'adresse 127.0.0.1 utilisée par mon serveur n'est pas accessible directement depuis Internet. Stripe ne pouvait donc pas envoyer directement cette requête vers mon application locale.

Pour reproduire cet échange en local, j'ai installé Stripe CLI. Cet outil permet d'écouter les événements Stripe puis de les transférer vers l'endpoint `/stripe/webhook` de mon application. Je me suis aidée de ChatGPT pour adapter les commandes à mon environnement Windows et à la configuration de mon projet.

Pendant les tests, le serveur de l'application et Stripe CLI doivent fonctionner simultanément dans deux terminaux. Lorsqu'un événement est généré par Stripe, Stripe CLI le reçoit puis le transmet à mon serveur local. Je peux également suivre dans le terminal les événements transférés et les réponses retournées par l'application.

J'ai vérifié le fonctionnement en effectuant des paiements et des remboursements en mode test, puis en contrôlant que les événements attendus arrivaient bien sur le webhook et entraînaient les changements de statuts correspondants dans l'application.

Cette mise en place m'a permis de mieux comprendre le fonctionnement d'un webhook et, plus généralement, la communication entre mon application et un service externe. Elle m'a également permis de comprendre la différence entre mon environnement local et un futur environnement de production. Une fois le site déployé sur une adresse publique, Stripe pourra envoyer directement ses événements vers l'URL publique du webhook. L'écouteur Stripe CLI utilisé pour effectuer le transfert vers mon serveur local ne sera alors plus nécessaire.

16. Bilan du projet et conclusion

Le projet Entre 2 Rives répond aux principaux besoins définis au début de sa conception. L'application permet de présenter l'activité et les différents modèles de bateaux, de consulter les tarifs et de rechercher les disponibilités selon une date et un nombre de passagers. Un utilisateur disposant d'un compte actif peut effectuer une réservation, la payer en ligne en mode test, consulter ses réservations et demander leur annulation lorsque les conditions prévues sont respectées.

La partie administration permet de suivre les réservations et les clients, de gérer les bateaux et les modèles ainsi que les formules et leurs tarifs. Le paiement et le remboursement sont intégrés avec Stripe et les confirmations sont traitées par webhook. Un formulaire de contact ainsi que les principales pages légales complètent les fonctionnalités du site.

La réalisation de ce projet m'a permis de mettre en pratique une grande partie des compétences abordées pendant la formation. J'ai travaillé sur la conception de la base de données, l'utilisation de Doctrine, les contrôleurs, les repositories et les services Symfony, mais également sur la réalisation des interfaces avec Twig, CSS, Bootstrap et JavaScript. Le projet m'a aussi amenée à travailler sur des aspects complémentaires comme la sécurité, les tests automatisés, l'intégration d'un service de paiement externe, l'accessibilité, le référencement et la prise en compte de la protection des données.

Au fur et à mesure du développement, j'ai également mieux compris l'intérêt de séparer les responsabilités dans l'application. Certaines règles métier, notamment celles concernant les disponibilités, les réservations ou les annulations, nécessitent plusieurs vérifications. Leur organisation dans des services et des repositories permet de conserver des contrôleurs plus simples et de rendre le fonctionnement général de l'application plus lisible.

L'application présente néanmoins encore certaines limites. La disponibilité des bateaux est actuellement calculée à partir des réservations, mais elle ne prend pas en compte le niveau de charge des batteries ni le temps éventuellement nécessaire à leur recharge entre deux locations. Dans le cadre d'une exploitation réelle de bateaux électriques, cette contrainte pourrait avoir une incidence sur les créneaux pouvant effectivement être proposés.

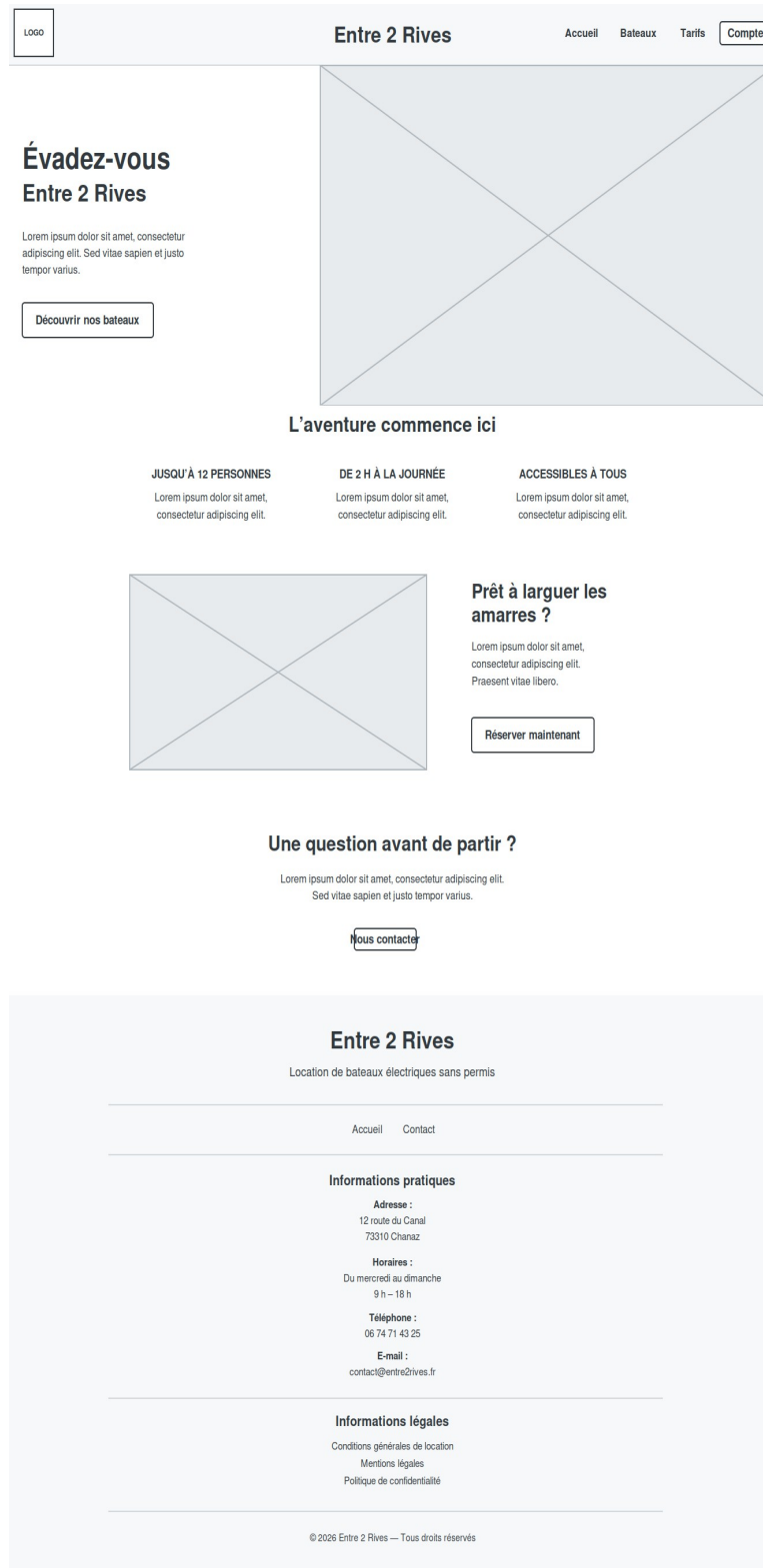
Une autre évolution concernerait l'espace client. L'utilisateur peut actuellement consulter ses informations personnelles, mais il ne peut pas les modifier directement depuis son compte. Une interface dédiée pourrait lui permettre de mettre à jour certaines données, tout en conservant les contrôles et les règles de sécurité nécessaires.

D'autres fonctionnalités pourraient être ajoutées dans le cadre d'une utilisation professionnelle réelle, mais l'objectif du projet était avant tout de réaliser une application cohérente couvrant l'ensemble du parcours principal, depuis la consultation des bateaux jusqu'à la réservation, au paiement et, si nécessaire, à l'annulation et au remboursement.

Ce projet m'a permis de construire une application complète à partir d'un besoin métier et de mieux comprendre les interactions entre les différentes parties d'une application web. Il m'a également permis de progresser dans l'organisation de mon code, dans la conception des traitements métier et dans ma capacité à rechercher et mettre en œuvre des solutions lorsque je rencontre une problématique nouvelle.

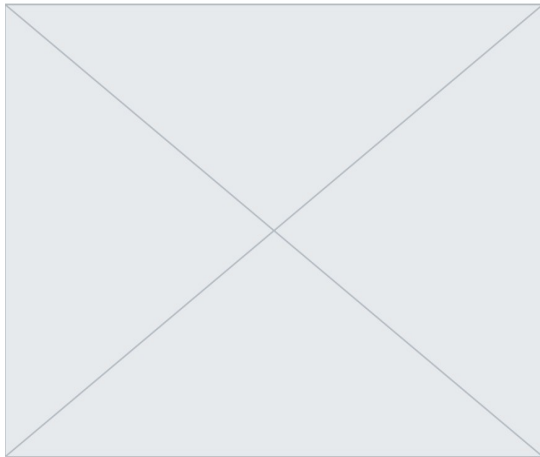
ANNEXES

Wireframes de la page d'accueil en format desktop, tablette, puis mobile



Évadez-vous Entre 2 Rives

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Praesent vitae sapien sed ligula tristique varius.
Vivamus tincidunt augue nec metus facilisis.

[Découvrir nos bateaux](#)

L'aventure commence ici

W

JUSQU'À 12
PERSONNES

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit.

DE 2 H À LA
JOURNÉE

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit.

ACCESSIBLES À
TOUS

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit.

,



Prêt à larguer les amarres ?

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Praesent vitae sapien sed ligula tristique varius.

[Réserver maintenant](#)

Une question avant de partir ?

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Praesent vitae sapien sed ligula tristique varius.

[Nous contacter](#)

Entre 2 Rives

Location de bateaux électriques sans permis

[Accueil](#)contact@entre2rives.fr

Informations pratiques

Adresse :

12 route du Canal
73310 Chanaz

Horaires :

Du mercredi au dimanche
9 h – 18 h

Téléphone :

06 74 71 43 25

E-mail :

contact@entre2rives.fr

Informations légales

Conditions générales de location

Mentions légales

Politique de confidentialité

© 2026 Entre 2 Rives — Tous droits réservés

Évadez-vous Entre 2 Rives

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Praesent vitae sapien sed
ligula tristique varius.

Découvrir nos bateaux



L'aventure commence ici

JUSQU'À 12 PERSONNES

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.

DE 2 H À LA JOURNÉE

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.

ACCESSIBLES À TOUS

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.



Prêt à larguer les amarres ?

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit.

Réserver maintenant

Une question avant de partir ?

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Praesent vitae sapien.

Nous contacter

Entre 2 Rives

Location de bateaux électriques
sans permis

Accueil

contact@entre2rives.fr

Informations pratiques

Adresse :

12 route du Canal
73310 Chanaz

Horaires :

Du mercredi au dimanche
9 h – 18 h

Téléphone :

06 74 71 43 25

E-mail :

contact@entre2rives.fr

Informations légales

Conditions générales de location
Mentions légales
Politique de confidentialité

© 2026 Entre 2 Rives — Tous droits réservés

Wireframe du catalogue en format desktop, tablette puis mobile

Nos bateaux

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Recherche par date et nombre de passagers

Date de location

Nombre de passagers

Rechercher

Effacer



Escapade

Jusqu'à 4 personnes

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip.

Voir ce bateau



Évasion

Jusqu'à 7 personnes

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip.

Voir ce bateau



Grand Large

Jusqu'à 12 personnes

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip.

Voir ce bateau

53

Nos bateaux

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Recherche par date et nombre de passagers

Date de location

Nombre de passagers

Rechercher

Effacer



Escapade

Jusqu'à 4 personnes

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip.

Voir ce bateau



Évasion

Jusqu'à 7 personnes

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip.

[Voir ce bateau](#)



Grand Large

Jusqu'à 12 personnes

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip.

[Voir ce bateau](#)

Recherche par date et nombre de
passagers

Date de location

jj / mm / aaaa

Nombre de passagers

Rechercher

Effacer



Escapade

Jusqu'à 4 personnes

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
Ut enim ad minim veniam, quis
nostrud exercitation ullamco.

Voir ce bateau



Évasion

Jusqu'à 7 personnes

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
Ut enim ad minim veniam, quis
nostrud exercitation ullamco.

Voir ce bateau



Grand Large

Jusqu'à 12 personnes

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
Ut enim ad minim veniam, quis
nostrud exercitation ullamco.

Voir ce bateau

